

MUHAMMAD ZAID'S

AI PRODUCT ENGINEER ROADMAP

AI Engineering · LLM Applications · FastAPI Backend · AI SaaS · Cloud
& Production · Flutter Integration

12-Week AI Engineering Sprint + 24-Week AI Product Engineer Expansion

*From proven mobile product shipper to an engineer who can design, build,
evaluate, deploy, and operate complete AI-powered products.*

OWNER	STARTING POINT	TARGET	FORMAT
Muhammad Zaid	Flutter product developer	AI Product Engineer	12-week sprint + 24-week expansion

Upgraded edition · Preserves the strongest material from Summer 2026 NLP/LLM Engineering Roadmap
v3

Prepared July 2026

EXECUTIVE BRIEF

Executive Brief

The target identity: AI Product Engineer

MISSION Become the engineer who can take a real user or business problem from requirements through AI architecture, FastAPI backend, data systems, AI application logic, cloud deployment, monitoring, Flutter integration, and real-user feedback.

Muhammad Zaid’s starting advantage

EXISTING PROOF	ROADMAP EXPANSION	CAREER DESTINATION
60+ mobile applications built; 30+ live on Google Play; remaining applications under review	Production LLM applications, RAG, agents, evaluation, and AI security	Strong junior / early-career AI Application or AI Product Engineer
Flutter, Dart, Firebase, Clean Architecture, state management, clients, and freelance delivery	FastAPI architecture, PostgreSQL, Redis, asynchronous jobs, SaaS controls, cloud, and operations	Credible foundation for Senior AI Engineer / Senior Software Engineer through years of real production ownership

Career direction: compound product strength with AI engineering



HONEST OUTCOME Completing this roadmap does not automatically make Muhammad Zaid a senior engineer or guarantee employment at a FAANG company. Successful completion should make him capable of competing for strong junior or early-career roles such as AI Application Engineer, LLM Application Engineer, AI Product Engineer, AI Backend Engineer, and Applied AI Engineer.

Progression toward senior engineering requires sustained production ownership, real users, team collaboration, code reviews, incidents and recovery, scaling experience, architecture decisions, mentorship, business impact, and several years of professional growth.

NAVIGATION

Table of Contents

Click any entry to jump to that section. Internal links remain active in the exported PDF.

FRONT MATTER

[Executive Brief](#)

[Role Definition](#)

[Operating System](#)

[Technology Decision Guide](#)

[Document Map](#)

[Pre-Week Setup](#)

[Part 1 - 12-Week Summer AI Engineering Sprint](#)

[Week 1 - LLM Applications](#)

[Week 2 - Embeddings and RAG](#)

[Week 3 - LangChain and LangGraph](#)

[Week 4 - Production RAG: Ingestion](#)

[Week 5 - Production RAG: Evaluation](#)

[Week 6 - Agent Tools and State](#)

[Week 7 - Agent Evaluation and Deployment](#)

[Week 8 - Measured Advanced Upgrade](#)

PART 1 - CONTINUED

[Week 9 - Production Hardening](#)

[Week 10 - n8n, WhatsApp, and Portfolio](#)

[Week 11 - Outreach and Internships](#)

[Week 12 - Delivery and Transition](#)

[Part 2 - 24-Week AI Product Engineer Expansion](#)

[Phase 1 - Professional FastAPI](#)

[Phase 2 - Databases, Jobs, and Pipelines](#)

[Phase 3 - AI SaaS Engineering](#)

[Phase 4 - Cloud and Production](#)

[Phase 5 - Flutter Integration](#)

[Phase 6 - Flagship Capstone](#)

[Production Engineering Standard](#)

[Final Portfolio](#)

[Competency Audit](#)

[Reference Index](#)

[Final Note](#)

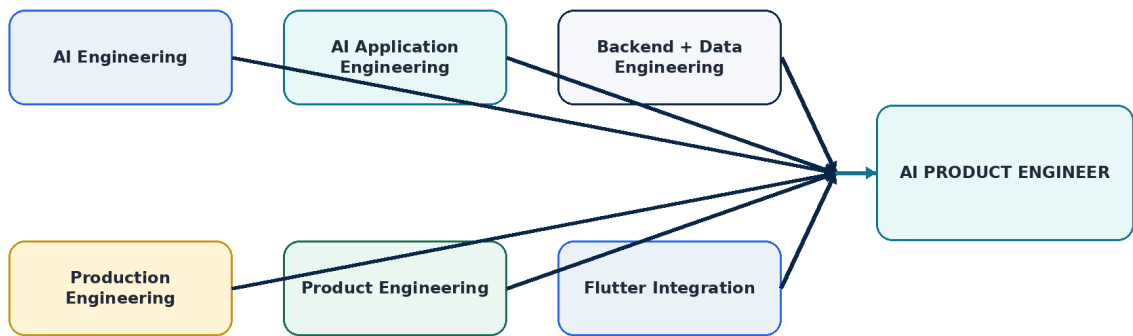
NAVIGATION STANDARD The roadmap advances through DONE WHEN gates. Use the Table of Contents for navigation, but use evidence - not page completion - to decide when to move forward.

ROLE DEFINITION

Role Definition

End-to-end AI product ownership

AI Product Engineer capability tree



Capability-layer definitions

ROLE	CONSISTENT DEFINITION
AI Engineer	Builds intelligent systems using LLMs, structured outputs, embeddings, RAG, agents, evaluation, security, and AI reliability.
AI Application Engineer	Integrates AI capabilities into usable software through APIs, backend services, databases, authentication, product features, and deployment.
AI Product Engineer	Owns the complete path from a user or business problem to requirements, architecture, implementation, evaluation, security, deployment, monitoring, user feedback, and product improvement.

What this role means

An AI Product Engineer does not stop at calling an LLM API. The role combines product judgment, AI application engineering, backend and data systems, evaluation, security, deployment, observability, and user feedback into one shipped outcome.

OWNS	DOES NOT MEAN
Problem framing, requirements, architecture, API/data boundaries, evaluation, deployment, and iteration	A UI-only chatbot, a prompt collection, or an API key embedded in a mobile application

OWNS	DOES NOT MEAN
Secure multi-user product behavior, observable operations, cost/reliability decisions, and real-user outcomes	Deep ML research by default, random framework collecting, or claiming seniority after tutorials

OPERATING SYSTEM

Operating System

How to execute this roadmap

MODE	SUMMER SPRINT	POST-SUMMER EXPANSION
Time budget	6 focused build days/week; roughly 36–42 focused hours; scheduled full rest days	15–20 focused hours/week alongside university; adjust scope before sacrificing health or coursework
Learning loop	Learn → build → test → measure → document each day	Design → implement → operate → gather feedback → improve each phase
Proof standard	Working repository, evaluation, demo, deployment, and retrospective	Multi-user product behavior, tests, operations, release evidence, and external users
Progress rule	Advance on DONE WHEN evidence, not because the calendar changed	Do not start Flutter until backend, data, SaaS, and cloud gates are complete
EXECUTION RULE When behind, reduce optional scope before reducing testing, evaluation, security, documentation, or rest.		

TOOL-SELECTION RULE Learn the concepts broadly, but implement one primary production path.

Do not attempt to master every model provider, vector database, cloud platform, worker system, and monitoring platform simultaneously.

Choose one main tool per responsibility, document the decision, and learn alternatives at awareness level.

Roadmap execution principles

1. Advance through evidence, not calendar completion.
2. Learn concepts before frameworks.
3. Implement one primary technology path.
4. Know alternatives and their trade-offs.
5. Reduce optional scope before reducing tests, evaluation, security, documentation, or rest.
6. Never claim a skill only because a tutorial was completed.
7. Do not use agents where deterministic code is safer.

8. Treat free tiers as learning constraints—not permanent production architecture.
9. Verify current models, APIs, pricing, quotas, privacy policies, and provider availability through official documentation.
10. Use Flutter only after the backend and AI platform are production-capable.

Primary implementation path

RESPONSIBILITY	PRIMARY IMPLEMENTATION PATH	ALTERNATIVES / DECISION BOUNDARY
AI + backend language	Python	Use as the primary AI and backend language; retain Dart for the final Flutter client.
Backend framework	FastAPI	The professional API and orchestration boundary.
Relational source of truth	PostgreSQL	Authoritative users, workspaces, permissions, product records, and usage ledger.
Temporary state + coordination	Redis	Caching, sessions, rate limits, temporary state, and job coordination—not durable truth.
Vector search	ChromaDB locally; choose Qdrant or pgvector for production	Select one production option based on filtering, operations, scale, and architecture.
Object storage	One S3-compatible or provider-native service	Original uploads and generated artifacts; do not store large files in relational rows.
Agent workflows	LangGraph	LangChain supports integrations and tooling; it is not the primary workflow-control layer.
Containerization + delivery	Docker + GitHub Actions	One reproducible container path and one CI/CD workflow.
Cloud	One practical provider during implementation	Learn other providers at awareness level instead of splitting execution across all of them.
Mobile integration	Flutter in Weeks 29–31	Use only after AI, backend, data, SaaS, and cloud phase gates are complete.

Evidence-first engineering loop

- (1) Define the user/business problem and acceptance criteria.
- (2) Implement the smallest end-to-end vertical slice.
- (3) Create tests and a versioned evaluation dataset before tuning.
- (4) Measure quality, latency, cost, reliability, and security behavior.
- (5) Diagnose root causes; change one major variable at a time.
- (6) Deploy, observe real usage, document trade-offs, and decide the next step.

Provider and cost strategy

DURABLE RULE Free tiers are a learning constraint—not a production architecture. Model names, quotas, pricing, and privacy terms change. At each phase, choose a current stable model by capability, structured-output/tool support, latency, privacy, and measured cost. Keep provider/model configuration outside business logic and log usage from Day 1.

LAYER	LEARNING DEFAULT	PROFESSIONAL EXPECTATION
LLM providers	Gemini plus Groq or OpenRouter for adapter/switching practice	Capability-based model routing, explicit fallback, budgets, privacy review, and no hard-coded provider
Vector store	ChromaDB locally	Qdrant or pgvector selected through scale, filters, operations, and team constraints
Observability	Structured logs plus LangSmith traces	Trace IDs, latency, tokens/cost, errors, task success, redaction, and operator alerts
Deployment	Low-cost learning environment	Environment separation, managed data, secrets, backups, CI/CD, rollback, and monitoring

ENGINEERING DECISIONS

Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs

DECISION PRINCIPLE Know the alternatives, but implement one primary path. Understanding why a tool was selected is more important than collecting every framework.

No tool is universally best. Select technology from product requirements, existing team skills, scale, cost, privacy, infrastructure, operational complexity, vendor constraints, and time to market. Alternatives remain decision literacy—not extra mandatory learning tracks.

Core application and data stack

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
AI and backend language	Python; retain Dart for Flutter	TypeScript; Java; C#; Go; Rust	Strongest AI/LLM and data ecosystem; FastAPI fit; rapid iteration with production support	TypeScript for Node/web teams; Java or C# for enterprise platforms; Go for high-throughput services; Rust for low-level safety/performance; Dart for the client	Implement deeply: Python + Dart; Working awareness: others
Backend framework	FastAPI	Django/DRF; Django Ninja; Flask; Litestar; NestJS; ASP.NET Core; Spring Boot	Python-native AI integration, Pydantic, OpenAPI, async, streaming, and a clear path from service to production API	Django for admin-heavy monoliths; NestJS for TypeScript teams; ASP.NET or Spring for established enterprise ecosystems; Flask for very small/legacy services	Implement deeply: FastAPI; Working awareness: alternatives
API style	REST + Server-Sent Events	WebSockets; GraphQL; gRPC; polling; webhooks	Clear resource contracts, broad client support, OpenAPI, straightforward testing, and strong Flutter integration; SSE fits one-way token/progress streams	WebSockets for bidirectional realtime; polling for simple job status; webhooks for server events; gRPC for internal high-performance calls; GraphQL for justified client-controlled queries	Implement deeply: REST + SSE; one WebSocket use case; Working awareness: GraphQL/gRPC

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Validation and ORM	Pydantic + SQLAlchemy + Alembic	SQLModel; Django ORM; Tortoise ORM; Prisma; raw SQL; SQLAlchemy Core	Typed API contracts, mature persistence, controlled migrations, and separation between transport and database models	SQLModel for smaller/simple services; Django ORM with Django; Prisma for TypeScript; raw SQL/Core for specialized or performance-critical queries	Implement deeply; include raw SQL and query-plan awareness
Relational source of truth	PostgreSQL	MySQL; MariaDB; SQLite; SQL Server; Oracle; MongoDB; DynamoDB; Firestore	Transactions, constraints, JSON, indexing, mature operations, pgvector option, and strong SaaS fit	SQLite for local/single-user tools; existing MySQL/enterprise databases when organizationally standard; document/managed NoSQL for intentionally designed non-relational access patterns	Implement deeply: PostgreSQL; Working awareness: NoSQL boundaries
Cache, sessions, limits, temporary state	Redis—never the authority for users, permissions, or billing	Memcached; database-backed sessions; in-memory cache; DynamoDB; managed caches	One proven system for cache, revocation state, rate counters, locks, temporary workflow state, and queue coordination	In-memory only for single-process development; Memcached for simple cache; database sessions when low traffic and infrastructure simplicity dominate	Implement deeply for defined temporary responsibilities

AI data, retrieval, workflow, and model systems

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Background jobs and workers	Redis + Celery default; implement one worker stack	Dramatiq; RQ; ARQ; FastAPI BackgroundTasks; RabbitMQ; Kafka; cloud queues; Temporal	Mature retries, scheduling, worker management, and broad production usage	BackgroundTasks only for small non-durable work; RQ/Dramatiq for simpler queues; RabbitMQ for routing/delivery semantics; Kafka for event streams; Temporal for durable long-running workflows; managed queues to reduce operations	Implement deeply: one stack; Working awareness: Kafka/Temporal

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Vector search	ChromaDB locally; choose pgvector or Qdrant for production	Pinecone; Weaviate; Milvus; Elasticsearch/OpenSearch; Vespa; Redis Vector Search	Chroma removes local infrastructure; pgvector simplifies a PostgreSQL-centered stack; Qdrant provides dedicated vector operations and strong filtering	Managed Pinecone to reduce operations; Weaviate for its integrated hybrid model; Milvus for larger specialist workloads; Elasticsearch/OpenSearch when keyword/vector search already lives there	Implement deeply: one production choice; Working awareness: others
RAG framework	Direct Python components first; LangChain selectively	LlamaIndex; Haystack; Semantic Kernel; custom framework-independent implementation	Exposes parsing, chunking, embedding, retrieval, ranking, generation, and failure modes before abstractions; LangChain then accelerates integration	LlamaIndex for connector/index workflows; Haystack for explicit search pipelines; Semantic Kernel for Microsoft ecosystems; custom code for strict control/minimal dependencies	Implement deeply: direct pipeline + selective LangChain; Working awareness: alternatives
Agent and workflow control	LangGraph	Direct Python state machine; LlamaIndex Workflows; Semantic Kernel; AutoGen; CrewAI; PydanticAI; Temporal; deterministic services	Explicit state, nodes, edges, stop/retry controls, human review, and traceable bounded workflows	Deterministic code whenever safer; direct state machines for small flows; PydanticAI for typed Python agents; Temporal for durable business workflows; multi-agent frameworks only for justified experiments	Implement deeply: LangGraph + deterministic boundaries; Working awareness: multi-agent tools
LLM providers	Provider-neutral adapter; measured hosted primary + fallback	OpenAI; Gemini; Anthropic; Groq; OpenRouter; Azure/AWS-hosted models; Ollama/vLLM local models	Keeps business logic portable and selects models from evaluation quality, structured output, tools, latency, cost, privacy, availability, and reliability	Local models for privacy/offline/control/high sustained volume; Azure/AWS for enterprise constraints; any hosted provider when it wins the product evaluation and policy review	Implement deeply: adapter + two hosted providers; Future specialization: local serving
Model serving	Hosted provider APIs	Ollama; vLLM; Hugging Face TGI; managed endpoints; Kubernetes inference	Fast iteration without GPU operations keeps focus on application engineering and measured provider comparison	Self-host for data residency, privacy, predictable high volume, specialized models, or infrastructure control	Implement deeply: hosted APIs; Future specialization: self-hosting

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
AI evaluation	Custom versioned harness + pytest	Ragas; DeepEval; Phoenix; LangSmith evaluations; Promptfoo; TruLens; human review	Product-specific expected behavior, version control, reproducibility, and no dependency on one evaluator library	Add one tool when it improves RAG metrics, prompt/provider comparison, trace-based runs, or review workflow; retain human judgment for subjective quality	Implement deeply: custom harness; Working awareness: tools
AI observability	Structured logs + trace IDs + OpenTelemetry concepts + one AI tracer	LangSmith; Langfuse; Phoenix; Helicone; Datadog; New Relic; Grafana/Prometheus; Sentry	Combines application health with prompt, retrieval, tool, token, latency, cost, evaluation, and outcome evidence	Choose the AI tracer and operations platform that match framework, privacy, team, and cloud; add Sentry/equivalent for capstone errors	Implement deeply: logs/trace IDs + one tracer; Working awareness: OTel/platforms

Identity, storage, cloud, and delivery platform

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Authentication and authorization	Learn JWT/OAuth2 through FastAPI; capstone chooses custom JWT or one managed identity provider	Firebase Auth; Auth0; Clerk; Cognito; Supabase Auth; Keycloak; enterprise identity providers	Builds token, refresh, expiry, revocation, role, permission, and ownership understanding while preserving a practical capstone choice	Managed identity for faster secure delivery, verification, reset, social login, and MFA; Firebase is attractive given existing Flutter experience. Backend authorization remains mandatory	Implement deeply: concepts + deterministic authorization; Working awareness: managed providers
Object storage	S3-compatible concept; local/MinIO in development; one managed provider in production	Amazon S3; Google Cloud Storage; Azure Blob; Cloudflare R2; MinIO; Supabase Storage; Firebase Storage	Correct home for original uploads, derived artifacts, exports, versions, and large binary objects without bloating PostgreSQL	Choose the provider already aligned with cloud, region, egress, cost, compliance, and Flutter/backend needs	Implement deeply: one managed provider; Working awareness: alternatives

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Cloud platform	One simple beta platform if needed; one major cloud for production	AWS; Google Cloud; Azure; Render; Railway; Fly.io; DigitalOcean; managed containers/serverless	A single complete path builds deployment and operations skill without splitting effort across providers	Simple platforms optimize time to market; AWS for breadth; GCP for data/AI and Cloud Run; Azure for Microsoft enterprise identity/integration	Implement deeply: one path; Working awareness: other clouds
Containers and orchestration	Docker + Docker Compose	Podman; cloud-native builds; serverless functions; Kubernetes; Nomad	Reproducible local and CI environments for API, worker, PostgreSQL, Redis, and supporting services with broad cloud portability	Kubernetes only for real multi-service scale, availability, and team capacity; serverless for suitable event/stateless workloads; Podman where organizationally preferred	Implement deeply: Docker/Compose; Working awareness: Kubernetes fundamentals
Infrastructure as code	Terraform or OpenTofu for one staging exercise	Pulumi; CloudFormation; AWS CDK; Azure Bicep; provider templates	Declarative, reviewable, reproducible infrastructure and transferable provider concepts	Pulumi for general-purpose languages; CloudFormation/CDK for AWS-only teams; Bicep for Azure-only teams	Working awareness + one small implementation; Future specialization after real cloud use
CI/CD	GitHub Actions	GitLab CI; Azure DevOps; CircleCI; Jenkins; Bitbucket Pipelines; cloud-native pipelines	Direct repository integration and accessible test, scan, image-build, and deployment automation for portfolio work	Use the platform already owned by the organization; Jenkins for established highly customized infrastructure	Implement deeply: one complete pipeline; Working awareness: transferable principles

Cross-cutting product, security, commercial, and client decisions

RESPONSIBILITY	PRIMARY CHOICE	MAIN ALTERNATIVES	WHY PRIMARY WAS CHOSEN	WHEN AN ALTERNATIVE IS BETTER	LEARNING DEPTH
Security	Deterministic backend authorization + OWASP API/LLM guidance + defense in depth	Managed security/cloud controls complement but do not replace application controls	Protects client, API, identity, database, object storage, vector retrieval, tools, prompts, logs, and cloud layers; the model never makes final authorization decisions	Provider-native controls are useful when they strengthen the same least-privilege, isolation, audit, file/output validation, and secret-management design	Implement deeply across every phase
Payments and billing	Mock entitlements, quotas, and usage ledger first; choose a real provider only after current verification	Stripe; Paddle; Lemon Squeezy; regional/local providers; App Store/Play Store billing	Separates product access logic from a provider whose country support, payouts, tax handling, registration, and store rules can change	Select the provider that currently supports the business country, product, tax model, payout needs, and mobile-store obligations	Implement deeply: ledger/entitlements; Future specialization: real billing integration
Flutter client architecture	Flutter as the final client	React Native; native Android/iOS; React/Next.js web; desktop clients	Builds on proven Flutter shipping strength and accelerates mobile integration after the AI/backend platform is production-capable	Use another client when target users, team skills, platform-native features, or web-first distribution demand it. Never place provider keys, secret prompts, authorization rules, credentials, or privileged tools in the client	Implement deeply: Flutter integration ; Working awareness: alternatives

IDENTITY CONTROL CHECKPOINT The selected authentication approach must support email verification, password reset, session or device revocation, account deletion, role-based authorization, and resource-level authorization. Managed authentication can accelerate identity delivery, but deterministic backend code still owns authorization and resource access.

OBSERVABILITY LAYERS Application observability covers logs, metrics, traces, errors, uptime, queue health, and database health. AI observability covers prompts, provider/model configuration, tokens, latency, retrieval results, tool calls, evaluations, cost, and task outcome. AI traces do not replace backend and infrastructure monitoring.

COMMERCIAL CONTROL CHECKPOINT The product layer must understand plans, entitlements, usage quotas, a usage ledger, subscription lifecycle, webhooks, refund and cancellation state, and payment-provider reconciliation. Build these contracts with a mock provider first; select a real provider only after verifying current country, payout, tax, business-registration, and mobile-store support.

VOLATILE PROVIDER FACTS Model names, prices, quotas, rate limits, and privacy terms change. Verify current official documentation before every phase. Do not treat any free tier as permanent production infrastructure.

Architecture Decision Record standard

For every major implementation, create a short ADR and link it from the repository README. Revisit the decision when requirements, scale, cost, privacy, infrastructure, or vendor constraints materially change.

ADR FIELD	REQUIRED CONTENT
1 · Problem	The engineering or product decision that must be made
2 · Requirements	Functional, quality, privacy, scale, cost, operations, and delivery constraints
3 · Options considered	Viable primary and alternative technologies—not a random tool list
4 · Selected option	The one implementation path chosen for this product and phase
5 · Why selected	Evidence and trade-offs that connect the choice to requirements
6 · Trade-offs	Benefits deliberately gained and limitations deliberately accepted
7 · Risks	Failure, lock-in, skills, cost, migration, privacy, and operational risks
8 · Revisit when	Explicit trigger conditions for reconsidering the decision

DOCUMENT MAP

Document Map

Roadmap at a glance

Part 1 — 12-week Summer AI Engineering Sprint

DURATION	ENGINEERING FOCUS	PRIMARY OUTPUT	GATE
1-3	LLM applications, embeddings/RAG, LangChain/LangGraph, market grounding	Assistant, extractor, mini RAG, agentic RAG, two real briefs	Foundation proven
4-5	Production document intelligence	Deployed cited RAG system with evaluation and security	RAG gate
6-7	Production agent engineering	Deployed LangGraph agent with trajectory evaluation and API	Agent gate
8-9	Advanced upgrade and hardening	Measured upgrade, production controls, case study, user validation	Portfolio centerpiece
10	n8n/WhatsApp sprint and portfolio	Commercial prototype, portfolio hub, resume, LinkedIn	Market-ready evidence
11-12	Outreach, delivery, retrospective, transition	Opportunity pipeline and signed six-month plan	Expansion ready

Part 2 — 24-week AI Product Engineer Expansion

DURATION	ENGINEERING FOCUS	PRIMARY OUTPUT	GATE
Weeks 13-16	Professional FastAPI backend	Versioned, secure, streaming, documented, tested API	Backend gate
Weeks 17-20	PostgreSQL, Redis, jobs, data pipelines	Durable asynchronous ingestion and tenant-safe data platform	Data gate
Weeks 21-24	AI SaaS product architecture	Multi-user beta with onboarding, limits, analytics, feedback, and cost tracking	SaaS gate
Weeks 25-28	Cloud and production engineering	Containerized CI/CD deployment with monitoring, backups, rollback, and runbook	Production gate
Weeks 29-31	Flutter AI product integration	Resilient production-connected mobile AI client	Mobile gate
Weeks 32-36	Flagship AI SaaS + mobile capstone	Externally used, evaluated, secure, observable, documented product	Product gate

What is preserved and what is upgraded

PRESERVED FROM v3	PROFESSIONAL UPGRADE
LLM fundamentals, prompting, structured output, embeddings, ChromaDB, RAG, LangChain, and LangGraph	Provider adapters, streaming, token/cost telemetry, reliability patterns, hybrid retrieval, reranking, document lifecycle, and data quality
Three market-driven projects, evaluation harnesses, prompt-injection testing, provider switching, and LangSmith	Versioned golden datasets, trajectory evaluation, access control, privacy, failure injection, release gates, and operational metrics
FastAPI integration and deployment	Professional architecture, authentication/authorization, PostgreSQL, Redis, durable jobs, SaaS controls, cloud, CI/CD, monitoring, backups, and rollback
Portfolio, client outreach, internships, rest cadence, and DONE WHEN checkpoints	AI Product Engineer identity, flagship SaaS/mobile product, competency proof, and honest career outcome

PART 1 · PRE-WEEK SETUP

Pre-Week Setup

Remove friction before Day 1

DONE WHEN A clean repository can start locally, secrets are excluded, two provider smoke tests pass, and the evaluation/results folders already exist.

Complete setup before the first learning day. Keep it minimal and reproducible; do not spend the week comparing tooling or perfecting infrastructure.

FOCUS	REQUIRED SETUP	EVIDENCE
Repository	Create a private-to-public-ready repo with README, LICENSE if appropriate, <code>.gitignore</code> , <code>.env.example</code> , <code>src/</code> , <code>tests/</code> , <code>evals/</code> , <code>docs/</code> , and <code>scripts/</code> .	Clean clone/install succeeds
Python	Use a supported Python version, virtual environment, pinned dependencies/lockfile, formatter/linter, pytest, and type checking appropriate to the project.	CI-style local checks pass
Providers	Create Gemini and one OpenAI-compatible provider account; verify current models, quotas, pricing, and privacy terms in official docs.	Two provider smoke tests
Data + tracing	Install ChromaDB locally, configure privacy-safe structured logs, and enable LangSmith or equivalent tracing for development.	One redacted trace and vector query
Security	Use a secret manager or environment variables, enable secret scanning where available, and create synthetic/sample documents only.	No key or sensitive data in Git

FOCUS	REQUIRED SETUP	EVIDENCE
Progress	Create a weekly evidence checklist, evaluation-results folder, architecture-decision log, and retrospective template.	First empty templates committed

Suggested repository contract

- ☐ Every runnable project has one command for setup, one for tests, one for evaluation, and one for local start.
- ☐ Every AI call passes through one provider interface with model, timeout, retries, trace ID, and usage metadata.
- ☐ Every major experiment records dataset version, configuration, result, decision, and rollback path.
- ☐ Every public repository uses synthetic or explicitly permitted data and contains no production credentials.

PART 1 · SUMMER 2026

PART 1 — 12-WEEK SUMMER AI ENGINEERING SPRINT

Sprint scope and progression

PART 1 OUTCOME Build the AI application layer through LLM applications, structured output, embeddings, RAG, LangChain, LangGraph, agentic workflows, measured production hardening, automation, portfolio packaging, outreach, delivery, and retrospective evidence.

DURATION	ENGINEERING FOCUS	PRIMARY OUTPUT	GATE
Weeks 1–3	LLM applications, structured output, embeddings, RAG, LangChain, LangGraph, and agentic workflows	Assistant, extractor, mini RAG, and controlled agent workflow	Foundation evidence
Weeks 4–5	Production RAG and document intelligence	Deployed cited knowledge system with evaluation	RAG gate
Weeks 6–7	Production LangGraph agent engineering	Deployed bounded agent with trajectory evaluation	Agent gate
Weeks 8–9	Advanced measured upgrade and production hardening	Measured improvements and hardened release	Hardening gate
Week 10	n8n, WhatsApp automation, and portfolio packaging	Commercial prototype and market-facing evidence	Packaging gate
Weeks 11–12	Outreach, delivery, retrospective, and transition	Opportunity pipeline and Part 2 transition plan	Expansion ready

SUSTAINABLE WORKLOAD Six focused workdays each week, approximately 36–42 focused hours, and scheduled rest days. Scope reduction comes before rest removal.

PART 1 · FOUNDATION · LLM APPLICATION ENGINEERING

WEEK 1 — LLM fundamentals, provider APIs, prompting, and structured output

DONE WHEN You can implement a model call from memory, switch providers through configuration, stream safely, validate structured output, and explain the cost and failure behavior of each request.

Goal

Build a reliable, provider-aware LLM application layer instead of a collection of one-off API calls.

Why this matters

WHY THIS MATTERS Every later RAG, agent, and SaaS system depends on predictable model inputs, validated outputs, controlled cost, and graceful failure behavior.

What to learn and apply

- Tokens, context windows, sampling, temperature, top-p, output limits, and model limitations
- System prompts, few-shot prompting, prompt templates, structured outputs, and tool/function calling
- Gemini, Groq, OpenRouter, and OpenAI-compatible client patterns through a provider adapter
- Streaming, timeouts, retries, fallbacks, token accounting, and privacy-aware provider selection

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Tokens and context	• Explain tokenization, context limits, truncation, and why long context does not guarantee good retrieval or reasoning. • Create a one-page model-behavior note with three failure examples: missing context, ambiguous instructions, and hallucinated facts. • Define a per-request budget: maximum input tokens, output tokens, latency, and acceptable cost.
Day 2	Prompt contracts	• Write and compare five system prompts for the same task; record what changed and why. • Practice zero-shot, few-shot, role/context/instruction separation, and explicit refusal/no-answer behavior. • Turn the strongest prompt into a reusable template with versioned test cases.
Day 3	Provider adapter	• Create one LLM interface with provider, model, timeout, retry, and streaming configuration. • Run smoke tests through Gemini and one OpenAI-compatible endpoint such as Groq or OpenRouter. • Keep secrets in environment variables; never commit keys, raw client data, or provider logs containing sensitive content.

DAY	FOCUS	PRACTICAL TASKS
Day 4	Structured outputs	- Define Pydantic schemas for an information extractor and use native structured output or tool calling. - Test clean, messy, incomplete, multilingual, and zero-result inputs; reject malformed outputs instead of silently accepting them. - Measure schema-valid response rate across at least 20 cases.
Day 5	Reliability and cost	- Add timeout, bounded retry with backoff, fallback model, and explicit error categories. - Capture latency, input/output tokens, model, provider, request ID, and estimated cost for every call. - Implement a safe failure message that does not expose stack traces, prompts, or secrets.
Day 6	Build: AI assistant	- Build a streaming command-line AI assistant with conversation history, reset, cancellation, and provider switching. - Add a small prompt-injection test set and confirm system rules are not treated as user-controlled data. - Document the architecture and known limitations.
Day 7	Build: information extractor	- Build a structured extractor for action items, owners, deadlines, and confidence from messy meeting notes. - Add unit tests for schema validation and a provider smoke-test suite. - Push both tools with a professional README, example output, test results, and a two-minute demo.

Project gate

DELIVERABLE	MILESTONE
Repository 01 — LLM Assistant + Structured Information Extraction System	Provider-agnostic LLM application layer with streaming, validated outputs, reliability controls, and usage telemetry.

Testing requirements

- ☐ At least 20 golden extraction cases with 100% schema-valid responses
- ☐ Provider smoke tests, timeout/retry tests, empty-input tests, and prompt-injection probes
- ☐ No secrets in Git history; logs contain request metadata but not unnecessary sensitive content

Common mistakes

- Hard-coding model IDs, API keys, or provider-specific response shapes throughout the application
- Treating temperature as a universal quality control or trusting generated JSON without validation
- Logging full prompts and user data by default; building only the happy path

Evidence required for GitHub or portfolio

- ☐ Architecture diagram and provider-adapter interface
- ☐ Golden dataset, automated test output, schema-validity score, and latency/token summary
- ☐ README, screenshots or demo video, and a short technical retrospective

PART 1 · FOUNDATION · RAG ENGINEERING

WEEK 2 — Embeddings, vector databases, retrieval, and document handling

DONE WHEN You can explain why a retrieval failed, identify whether the root cause is parsing, chunking, embedding, filtering, ranking, or generation, and prove the fix with a re-run.

Goal

Understand and build the retrieval layer behind grounded, knowledge-based AI products.

Why this matters

WHY THIS MATTERS Most business AI products must answer from private documents or product data. Retrieval quality—not prompt cleverness—is usually the limiting factor.

What to learn and apply

- Embeddings, cosine similarity, semantic search, metadata, top-k retrieval, and filtering
- ChromaDB for local learning; Qdrant and pgvector responsibilities and trade-offs
- Fixed, recursive, and semantic chunking; document parsing for PDF, Word, Excel, and scanned files
- Retrieval evaluation, citations, grounded answers, and no-answer behavior

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Semantic search	• Embed a small corpus and compare semantic retrieval with exact keyword search on the same ten queries. • Explain similarity score limitations and why thresholds must be calibrated on real data. • Record three queries where semantic search wins and three where keyword search wins.

DAY	FOCUS	PRACTICAL TASKS
Day 2	Vector storage	- Store 100+ chunks in ChromaDB with document ID, source, page, section, tenant, and timestamp metadata. - Practice filtered retrieval and deletion/re-indexing by document version. - Write a decision note: local Chroma vs managed Qdrant vs PostgreSQL + pgvector.
Day 3	Chunking experiments	- Chunk one real document using fixed, recursive, and structure-aware strategies. - Inspect boundary failures, lost headings, tables, and duplicated context. - Select a baseline using evidence rather than a copied chunk-size value.
Day 4	Messy documents	- Process at least two formats and one difficult input: low-quality scan, table-heavy PDF, Word file, or spreadsheet. - Preserve source location and extraction warnings so citations can be traced back. - Quarantine corrupt or unsupported files instead of failing the entire batch.
Day 5	Retrieval evaluation	- Create 15–20 questions with expected answer facts and expected source documents. - Measure retrieval hit rate/recall at k; manually inspect false positives and false negatives. - Add explicit no-answer questions that are outside the corpus.
Day 6	Build: ingestion + retrieval	- Implement load → parse → clean → chunk → embed → index with deterministic IDs and metadata. - Add a query function with filters and a debug mode that exposes retrieved chunks and scores. - Run the golden queries before connecting generation.
Day 7	Build: cited Q&A	- Generate answers only from retrieved context and return answer, citations, and confidence/reason code in a validated schema. - Require an honest no-answer when evidence is insufficient. - Push the mini RAG system with a failure log and a plan for Week 4–5 productionization.

Project gate

DELIVERABLE	MILESTONE
Repository 02 — Mini Document Intelligence / RAG System	A traceable ingestion and retrieval pipeline that handles multiple document types and produces cited, grounded answers.

Testing requirements

- ☐ 15–20 golden questions, including unanswerable and metadata-filtered cases
- ☐ Retrieval hit rate/recall@k baseline plus manual citation verification
- ☐ Corrupt-file, duplicate-upload, empty-document, and deleted-document tests

Common mistakes

- Evaluating only final prose while hiding the retrieved evidence
- Using one clean PDF and assuming the ingestion pipeline is production-ready
- Dropping metadata, document versions, tenant filters, or no-answer behavior

Evidence required for GitHub or portfolio

- ☐ Chunking comparison and vector-store decision note
- ☐ Retrieval evaluation table, citation samples, and failure analysis
- ☐ Architecture diagram, README, test output, and recorded demo

PART 1 · FOUNDATION · AGENT WORKFLOWS

WEEK 3 — LangChain, LangGraph, agentic RAG, and market grounding

DONE WHEN You can build and debug the graph without copying a tutorial, explain why each transition exists, and show that the planned projects answer real user or business requirements.

Goal

Build a small stateful workflow that chooses actions deliberately, recovers from poor retrieval, and remains observable.

Why this matters

WHY THIS MATTERS Agents are useful when a task requires decisions, tools, state, or human handoff. A graph makes control flow, retries, and safety boundaries explicit.

What to learn and apply

- LangChain model/tool abstractions and create_agent fundamentals
- LangGraph state, nodes, edges, conditional routing, persistence, and retry guards
- Tool contracts, graders, query rewriting, human handoff, and trajectory tracing
- Market research that turns current client/job requirements into project acceptance criteria

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	LangChain foundations	- Wrap current providers behind LangChain's model interface and define two typed tools. - Test each tool independently before granting it to an agent. - Compare direct SDK calls with framework abstractions; document what the framework adds and hides.
Day 2	Agent harness	- Build a minimal create_agent workflow with two harmless tools and a strict system policy. - Inspect tool-selection traces and deliberately trigger a wrong-tool case. - Improve tool descriptions and re-test rather than adding more prompt text blindly.
Day 3	LangGraph state	- Define typed state for question, retrieved documents, generation, error, retry count, and user/tenant context. - Build a two-node retrieve → generate graph. - Persist only the state required for product behavior; classify what must never be stored.
Day 4	Conditional control	- Add relevance grading, query rewrite, retry, and stop conditions. - Set maximum attempts and return a transparent failure/handoff outcome instead of looping. - Test irrelevant retrieval and unavailable-provider paths.
Day 5	Market research	- Review 15–20 current AI/RAG/agent job briefs or product examples and extract repeated business problems and acceptance criteria. - Record required deliverables, data types, integrations, reliability expectations, and evidence buyers request. - Select one realistic RAG brief and one agent brief for Weeks 4–7; remove any confidential client details.
Day 6	Build: agentic RAG	- Implement route → retrieve → grade → rewrite-or-generate with traceable state transitions. - Use the provider adapter from Week 1 and retrieval debug mode from Week 2. - Add a maximum-step guard and explicit no-answer path.
Day 7	Test + explain	- Run good retrieval, bad retrieval, provider timeout, and out-of-scope questions. - Draw the graph and explain every node, state field, conditional edge, and failure outcome aloud. - Freeze the two project briefs and acceptance criteria for the build phase.

DAY	FOCUS	PRACTICAL TASKS
REST	Scheduled recovery	- Full day off: no tutorials, coding, portfolio work, or outreach. - Resume only after the project briefs and acceptance criteria are clear.

Project gate

DELIVERABLE	MILESTONE
Agentic RAG prototype + two market-grounded project briefs	A bounded LangGraph workflow with visible state, conditional retry, provider independence, and clear project specifications.

Testing requirements

- ☐ Node-level unit tests and four end-to-end failure scenarios
- ☐ Trace review proving maximum-step behavior and correct no-answer/handoff outcomes
- ☐ Written acceptance criteria for the RAG and agent projects

Common mistakes

- Using an agent for a deterministic workflow that would be safer as ordinary code
- Giving broad tools excessive permissions or omitting maximum-step limits
- Choosing projects from a generic idea list without a user, brief, or acceptance criteria

Evidence required for GitHub or portfolio

- ☐ LangGraph architecture image and annotated trace
- ☐ Failure-case test output and provider-switch evidence
- ☐ Redacted market-research matrix and two signed-off project briefs

PART 1 · BUILD · DOCUMENT INTELLIGENCE SYSTEM

WEEK 4 — Production RAG project — ingestion, indexing, and retrieval

DONE WHEN Documents can be uploaded, processed, versioned, queried, debugged, and deleted without manual cleanup, and retrieval quality is measured rather than assumed.

Goal

Turn the Week 2 prototype into a domain-specific, multi-document RAG product built against a real brief.

Why this matters

WHY THIS MATTERS Production RAG begins before generation: durable ingestion, traceable metadata, tenant-safe retrieval, and measurable ranking quality create the foundation for trustworthy answers.

SUMMER AUTHENTICATION BOUNDARY During the 12-week summer sprint, use a limited development identity, static demo workspace, or simple protected demo access where necessary.

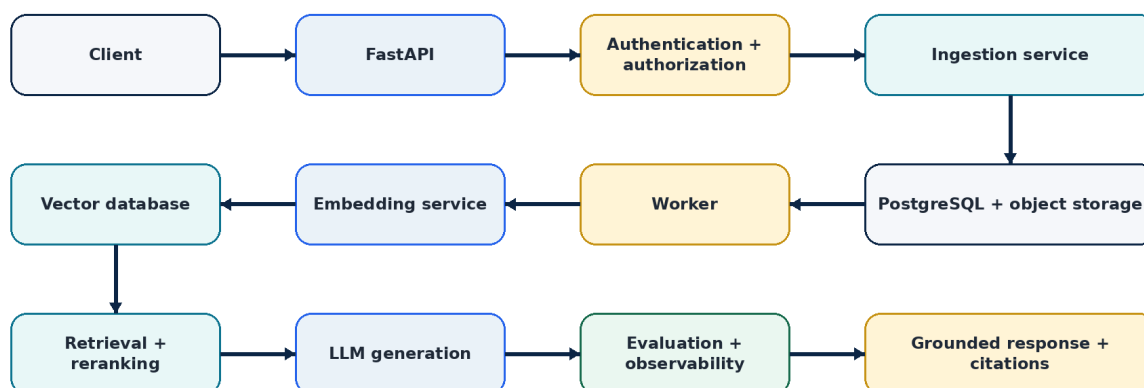
This is provisional and exists only to practice access boundaries.

Professional JWT authentication, role-based authorization, persistent users, workspace membership, and complete tenant isolation are implemented during Part 2 — Professional FastAPI and AI SaaS Engineering.

What to learn and apply

- Requirements, domain corpus design, document lifecycle, idempotent ingestion, and processing status
- Multi-format parsing, OCR, normalization, structure-aware chunking, metadata, and versioning
- Semantic + BM25 hybrid search, filtering, reranking, retrieval debugging, and vector-store choice
- Minimal FastAPI upload/status/query endpoints as a bridge—not yet the full backend curriculum

Production RAG architecture



Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Brief + acceptance criteria	- Choose the Week 3 RAG brief and write users, supported documents, queries, exclusions, privacy level, and success metrics. - Gather 5–20 representative documents with permission to use them. - Create an architecture decision record for Chroma, Qdrant, or pgvector.
Day 2	Document lifecycle	- Define document, file, processing job, page/section, chunk, embedding, and version identifiers. - Implement safe upload validation, duplicate detection, size/type limits, and processing states. - Store extraction warnings and failure reason codes.
Day 3	Parsing + cleanup	- Handle PDF, Word, and one of Excel or scanned OCR inputs. - Normalize whitespace while preserving headings, page references, tables, and source location. - Create parser fixtures for clean, messy, empty, password-protected, and corrupt documents.
Day 4	Chunking + metadata	- Run two chunking strategies on representative documents and compare retrieval outcomes. - Attach tenant/workspace, document, version, page, section, and access-scope metadata to every chunk. - Make re-index and delete operations deterministic.

DAY	FOCUS	PRACTICAL TASKS
Day 5	Hybrid retrieval	- Build vector search plus BM25 keyword search and combine scores using a documented method. - Add metadata filters and a reranking stage only after establishing the baseline. - Capture retrieved chunk IDs, ranks, scores, latency, and model version for debugging.
Day 6	Minimal API surface	- Expose upload, processing-status, list/delete document, and query endpoints through a small FastAPI layer. - Return stable response schemas and error codes; do not put retrieval logic inside routes. - Add a simple Streamlit or API-docs interface for demonstration.
Day 7	Baseline review	- Run the Week 2 golden questions against the new pipeline and record baseline retrieval metrics. - Inspect the ten worst cases and classify root cause before Week 5. - Freeze the baseline configuration and tag the release candidate.

Project gate

DELIVERABLE	MILESTONE
Production Document Intelligence System — ingestion and retrieval release candidate	Multi-document ingestion with citations-ready metadata, hybrid retrieval, reranking baseline, lifecycle operations, and a stable API boundary.

Testing requirements

- ☐ Parser fixture suite across at least three file types and one OCR/scanned case
- ☐ Idempotent re-upload, re-index, delete, corrupt-file, and metadata-filter tests
- ☐ Baseline retrieval metrics plus latency distribution and worst-case analysis

Common mistakes

- Starting with a reranker before establishing a reproducible baseline
- Losing source/page metadata or mixing documents across users/workspaces
- Running long ingestion synchronously inside the request path

Evidence required for GitHub or portfolio

- ☐ System architecture and ingestion sequence diagrams
- ☐ API contract, parser test report, retrieval baseline, and decision record
- ☐ README, sample documents, screenshots, and traceable query demo

PART 1 · BUILD · DOCUMENT INTELLIGENCE SYSTEM

WEEK 5 — Production RAG project — generation, evaluation, security, and deployment

DONE WHEN A user can upload permitted documents, receive evidence-backed answers, understand uncertainty, and trust that the system will not cross document or workspace boundaries.

Goal

Make the RAG system trustworthy enough for an external user by adding grounded generation, evaluation, security tests, observability, and deployment.

Why this matters

WHY THIS MATTERS A good demo can answer a few questions. A product must know when not to answer, cite evidence correctly, preserve access boundaries, survive failures, and show measurable improvement.

What to learn and apply

- Answer schemas, citation correctness, groundedness, relevance, completeness, and no-answer calibration
- Golden datasets, retrieval and answer evaluation, before/after experiments, and regression tests
- Prompt injection, document-level access control, output validation, secrets, privacy, and data leakage tests
- Tracing, latency/cost tracking, fallback behavior, deployment, user feedback, and retrospective

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Grounded generation	• Return validated answer, citations, no-answer reason, and trace ID; require each material claim to map to retrieved evidence. • Separate trusted system policy from untrusted document text and user instructions. • Add a response when sources disagree or evidence is incomplete.

DAY	FOCUS	PRACTICAL TASKS
Day 2	Golden dataset	- Expand to 30–40 representative questions: simple, multi-hop, ambiguous, conflicting, metadata-filtered, and unanswerable. - Store expected facts and source references separately from prompts. - Version the dataset and evaluation configuration in the repository.
Day 3	Evaluate + diagnose	- Measure retrieval success, groundedness, relevance, completeness, citation correctness, latency, and tokens/cost. - Classify every failure as parser, chunking, retrieval, ranking, prompt, model, or policy. - Change one variable at a time and publish before/after results.
Day 4	Security + privacy	- Test direct and indirect prompt injection embedded in user queries and documents. - Verify workspace/document access filters cannot be overridden by the model. - Redact sensitive logs, validate outputs, protect secrets, and document provider data-handling assumptions.
Day 5	Reliability + deployment	- Add retry/timeouts, provider fallback, rate/cost caps, health checks, and structured logs/traces. - Deploy the application and run the full evaluation suite against the deployed environment. - Write rollback and known-limitations notes.
REST	Scheduled recovery	- Full day off after the first production project release. - Do not replace rest with portfolio polishing or new features.
Day 7	Real user + case study	- Give one external tester a task list and observe without coaching. - Fix critical usability/reliability issues and re-run affected evaluations. - Publish a case study with problem, architecture, metrics, security notes, cost analysis, and retrospective.

Project gate

DELIVERABLE	MILESTONE
Portfolio Project 2 — Deployed Production RAG Knowledge System	Externally testable RAG product with cited answers, evaluation report, access controls, security probes, observability, and before/after metrics.

Testing requirements

- ☐ 30–40 versioned golden cases and regression results from the deployed system
- ☐ Citation verification, no-answer calibration, tenant/document isolation, and prompt-injection tests

- ☐ Timeout/fallback, rate/cost limit, health-check, and user-acceptance tests

Common mistakes

- Using an LLM judge as the only evaluator without human spot checks or source truth
- Optimizing answer style while retrieval and citation quality remain weak
- Deploying with sensitive test data, unrestricted uploads, or no cost/usage controls

Evidence required for GitHub or portfolio

- ☐ Live deployment, demo video, professional README, and API documentation
- ☐ Versioned evaluation report with before/after metrics and failure categories
- ☐ Security/privacy notes, cost per query, external-user feedback, and retrospective

PART 1 · BUILD · LANGGRAPH AGENT PLATFORM

WEEK 6 — Production agent project — tools, state, planning, and recovery

DONE WHEN The agent completes its defined task, cannot exceed its permissions, never loops indefinitely, and produces a trace that explains every action and recovery decision.

Goal

Build a tool-using agent for a real task with explicit permissions, state transitions, recovery paths, and human escalation.

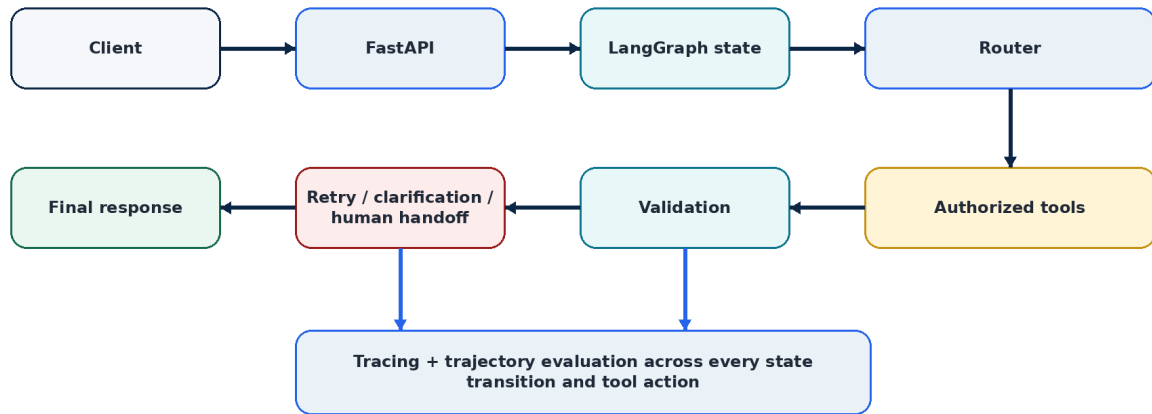
Why this matters

WHY THIS MATTERS Business automation succeeds only when tool contracts, permissions, failure handling, and final outcomes are controlled. Free-form autonomy is not a production architecture.

What to learn and apply

- Agent use-case selection, deterministic vs agentic boundaries, tool schemas, permissions, and idempotency
- LangGraph state, nodes, conditional edges, planning, short-term memory, and checkpointing
- Retries, timeouts, maximum steps, clarification, tool failure recovery, and human handoff
- Tracing and audit events for decisions, actions, outcomes, and denied operations

Controlled agent architecture



Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Brief + threat boundary	- Choose the Week 3 agent brief: support, business automation, or research assistant with a measurable task outcome. - List what the agent may read, write, call, and never do; identify which actions require confirmation. - Define success, safe failure, human handoff, maximum steps, latency, and cost budget.
Day 2	Tool contracts	- Implement 3–4 narrow typed tools with clear descriptions, validated arguments, least privilege, and deterministic return types. - Add idempotency keys or dry-run behavior for side-effecting actions. - Test tools independently with valid, invalid, unavailable, and unauthorized inputs.
Day 3	Graph state	- Design state for request, user/role, plan, tool results, retry count, error, confirmation, and final outcome. - Build router, tool, validator, response, and handoff nodes. - Persist only necessary conversation/task state with explicit retention rules.
Day 4	Planning + control	- Use a plan only for tasks that need decomposition; keep deterministic steps in code. - Add conditional edges for clarification, confirmation, tool selection, validation, retry, and stop. - Prevent the model from inventing tools or bypassing authorization.

DAY	FOCUS	PRACTICAL TASKS
Day 5	Failure recovery	- Inject timeouts, malformed tool data, partial results, duplicated actions, and provider failure. - Implement bounded retry, fallback, compensation/dry-run where appropriate, and human handoff. - Return stable failure categories that a UI or support operator can act on.
Day 6	Memory + observability	- Separate conversation history, task state, durable user facts, and retrieval memory; do not call all stored text 'memory.' - Trace node transitions, tool inputs/outputs, latency, token use, errors, and final outcome in LangSmith or equivalent tracing. - Redact secrets and sensitive tool payloads from traces.
Day 7	Integration baseline	- Run ten realistic scenarios and review each trajectory manually. - Freeze the graph, tool contracts, safety policy, and baseline task-success score for Week 7. - Create the architecture diagram and API-facing result schema.

Project gate

DELIVERABLE	MILESTONE
LangGraph agent release candidate with 3–4 tools, permissions, recovery, and human handoff	A bounded, traceable agent workflow that performs a real task and fails safely under tool, model, or user ambiguity.

Testing requirements

- ☐ Standalone tool tests plus authorization, invalid-argument, timeout, and duplicate-action scenarios
- ☐ Ten baseline end-to-end trajectories reviewed for tool order, outcome, and safety
- ☐ Maximum-step, clarification, confirmation, and human-handoff tests

Common mistakes

- Giving the model direct database or broad external-system access
- Treating chat history as durable memory without retention, privacy, or relevance rules
- Adding more agents when a single workflow or deterministic service would be clearer

Evidence required for GitHub or portfolio

- ☐ LangGraph diagram, tool-contract documentation, and permission matrix
- ☐ Baseline trajectory report, failure-injection results, and redacted traces
- ☐ README, demo scenario, cost/latency summary, and known limitations

PART 1 · BUILD · LANGGRAPH AGENT PLATFORM

WEEK 7 — Production agent project — evaluation, security, FastAPI bridge, and deployment

DONE WHEN The deployed agent completes the intended business task, every action is authorized and traceable, failures recover or hand off, and evaluation proves improvement over the baseline.

Goal

Prove agent reliability through trajectory evaluation, expose the workflow through a stable API, and deploy an auditable release.

Why this matters

WHY THIS MATTERS Agent quality is the path taken, not only the final sentence. Production systems must evaluate decisions, restrict actions, expose failures, and integrate with other products.

What to learn and apply

- Golden scenarios, task success, tool selection/order, trajectory quality, and human review
- Prompt injection, tool authorization, output validation, confirmation gates, and data leakage prevention
- FastAPI route/service separation, streaming, stable schemas, trace IDs, and error handling
- Retries, timeouts, fallbacks, cost ceilings, LangSmith observability, deployment, and runbooks

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Trajectory dataset	• Create 20–25 scenarios with expected tools, order constraints, required confirmation, prohibited actions, and expected outcome. • Include ambiguous requests, unauthorized users, missing data, tool failure, and handoff cases. • Version the dataset and define pass/fail rules before tuning the agent.
Day 2	Agent evaluation	• Score task success, correct tool selection, argument validity, prohibited-action rate, steps, latency, and cost. • Review failed trajectories and classify tool description, routing, state, policy, model, or integration causes. • Change one variable at a time and capture before/after results.

DAY	FOCUS	PRACTICAL TASKS
Day 3	Security tests	• Test direct prompt injection, malicious tool arguments, role bypass, data exfiltration attempts, and instructions hidden in retrieved content. • Enforce authorization in code before every tool call; never rely on the model to decide access. • Require confirmation for material external actions and validate the post-action result.
Day 4	FastAPI bridge	• Expose the agent through versioned request, status, cancel, and streaming-result endpoints. • Keep routes thin and place orchestration in a service layer; return trace ID and stable error codes. • Add API authentication for the demo and verify the core workflow works without Streamlit.
Day 5	Observability + cost	• Track node/tool latency, model/provider, tokens, cost, retries, failure category, and final task outcome. • Add alerts or visible thresholds for runaway steps, repeated tool failure, and cost-budget breach. • Write a short operator runbook for common failures.
REST	Scheduled recovery	• Full day off after the second production project release. • No portfolio work, outreach, or 'quick' feature additions.
Day 7	Deploy + case study	• Deploy the API and a thin demo interface; run evaluations against the deployed release. • Record a demo showing successful execution, denied action, tool failure recovery, and human handoff. • Publish architecture, API docs, evaluation results, cost/run, security notes, and retrospective.

Project gate

DELIVERABLE	MILESTONE
Portfolio Project 3 — Deployed LangGraph AI Agent	API-accessible agent with trajectory evaluation, authorization gates, bounded retries, observability, deployment, and operator documentation.

Testing requirements

- ☐ 20–25 versioned trajectories with success, tool-order, safety, latency, and cost results
- ☐ Authorization matrix, prompt-injection probes, confirmation, cancellation, timeout, and handoff tests
- ☐ Direct API, streaming, deployment smoke, and runbook recovery checks

Common mistakes

- Grading only the final response while ignoring wrong or dangerous tool calls

- Putting orchestration code directly inside the FastAPI route
- Exposing internal traces or tool payloads to users without redaction

Evidence required for GitHub or portfolio

- ☐ API documentation, deployed demo, architecture diagram, and operator runbook
- ☐ Before/after trajectory evaluation report and representative redacted traces
- ☐ Security test matrix, cost-per-successful-task analysis, and technical retrospective

PART 1 · BUILD · ADVANCED RAG OR AGENT UPGRADE

WEEK 8 — Market-directed advanced upgrade — deepen the strongest project

DONE WHEN The upgrade is retained because measured product value exceeds complexity and operating cost—not because it sounds advanced.

Goal

Select one high-value production upgrade based on evidence from real requirements and improve the strongest project without scope creep.

Why this matters

WHY THIS MATTERS Depth is more credible than a third shallow demo. A measured upgrade demonstrates engineering judgment, experimentation, and the ability to improve a working system.

What to learn and apply

- Upgrade selection: hybrid search + reranking, multi-document ingestion, provider routing, tenant isolation, or advanced recovery
- Baseline locking, design decision records, controlled experiments, and regression prevention
- Multi-agent concepts only when independent roles and coordination are genuinely required
- Failure injection, latency/cost trade-offs, and product-visible improvement

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Choose one upgrade	- Use market research, evaluation failures, and user feedback to rank upgrades by business impact, evidence strength, and effort. - Choose exactly one primary upgrade and define an explicit non-goal list. - Lock baseline metrics and write the target improvement before coding.
Day 2	Design	- Write an architecture decision record with options, chosen approach, risks, rollback, and success measures. - Create an experiment plan that changes one major variable at a time. - Add any new fixtures or dataset labels needed for fair evaluation.
Day 3	Build — part 1	- Implement the smallest vertical slice of the upgrade behind a configuration or feature flag. - Preserve the old path for comparison and rollback. - Add component-level tests before connecting it to the full workflow.
Day 4	Build — part 2	- Integrate the upgrade with logging, tracing, access filters, and error handling. - Run a small smoke subset and correct integration defects. - Document any new operational dependency or cost.
Day 5	Controlled evaluation	- Run baseline and upgraded configurations on the same dataset and environment. - Compare quality, latency, token use, cost, and failure rate—not quality alone. - Reject the upgrade if evidence does not justify its complexity.
Day 6	Failure injection	- Test degraded retrieval, unavailable reranker/provider, partial documents, tool failure, and budget exhaustion as applicable. - Verify the system falls back or fails transparently without corrupting state. - Add regression cases for every discovered failure.
Day 7	Catch-up + decision	- Complete only critical gaps; do not add a second upgrade. - Make a keep/revise/revert decision based on evidence. - Write the case-study outline for Week 9.

Project gate

DELIVERABLE	MILESTONE
Measured advanced upgrade to the strongest RAG or agent project	One production-relevant capability added behind a safe rollout path with objective before/after evidence.

Testing requirements

- ☐ Same-dataset baseline vs upgraded evaluation with quality, latency, cost, and failure comparison
- ☐ Feature-flag/rollback test and degraded-dependency scenarios
- ☐ Regression tests for every newly discovered failure

Common mistakes

- Adding hybrid search, multi-agent orchestration, and provider routing simultaneously
- Comparing results on different datasets or changing multiple variables at once
- Keeping an upgrade that improves one metric while badly harming latency, cost, or reliability

Evidence required for GitHub or portfolio

- ☐ Architecture decision record and experiment plan
- ☐ Before/after evaluation table with trade-off analysis
- ☐ Feature flag/rollback evidence and updated case-study assets

PART 1 · BUILD · PRODUCTION READINESS

WEEK 9 — Production hardening, case study, and real-user validation

DONE WHEN A reviewer can understand the problem, architecture, evidence, trade-offs, and limits in under ten minutes and can run or access a stable release.

Goal

Turn the strongest project into the portfolio centerpiece through operational hardening, user validation, and a rigorous technical case study.

Why this matters

WHY THIS MATTERS Recruiters, clients, and future teammates need evidence that the system can be operated, debugged, and improved—not only that it runs in a controlled demo.

What to learn and apply

- Configuration, structured errors, rate/cost limits, caching, health checks, logging, tracing, and basic load testing
- API contracts, security review, deployment checks, rollback, incident thinking, and known limitations
- Case-study writing, architecture communication, demo design, and user feedback
- Portfolio evidence that connects business problem, engineering decisions, metrics, and outcome

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Configuration + errors	• Remove hard-coded models, paths, URLs, and secrets; validate configuration at startup. • Define stable user-facing and operator-facing error categories with trace IDs. • Confirm logs are structured and privacy-safe.
Day 2	Operational controls	• Add rate/cost limits, timeouts, retries, fallbacks, health/readiness checks, and safe cancellation. • Use caching only for justified read paths; never cache across users or permissions accidentally. • Write rollback and dependency-failure procedures.
Day 3	Performance check	• Run a modest concurrent load test against representative requests. • Measure p50/p95 latency, errors, tokens/cost, and dependency bottlenecks. • Set product-specific service targets and document what is not yet scalable.
Day 4	Security review	• Review authentication boundary, authorization, tenant filters, file validation, prompt injection, output validation, and secret handling. • Re-run the security regression suite and close high-severity issues. • Document residual risks honestly.
Day 5	Case study	• Write problem → requirements → architecture → experiments → results → security → cost → limitations → next steps. • Create clean diagrams, API examples, evaluation charts/tables, and a five-minute demo. • Make the README understandable without opening the source code.
REST	Scheduled recovery	• Full day off before the outward-facing phase. • Return with enough energy for user conversations, portfolio work, and outreach.

DAY	FOCUS	PRACTICAL TASKS
Day 7	Real-user validation	• Give 3–5 target users or peers realistic tasks and capture completion, confusion, failures, and feedback. • Fix critical issues, re-run affected tests, and update the case study. • Tag a stable portfolio release.

Project gate

DELIVERABLE	MILESTONE
Portfolio centerpiece — hardened RAG or agent product with case study	A stable, measured, externally tested release with operator controls, security review, and a decision-quality engineering narrative.

Testing requirements

- ☐ Regression, security, dependency-failure, and modest concurrent-load results
- ☐ Health/readiness, rollback, configuration, log-redaction, and access-isolation checks
- ☐ 3–5 user sessions with issues traced to fixes and re-tested

Common mistakes

- Polishing the interface while critical reliability or access-control issues remain
- Claiming production scale without load, failure, or operational evidence
- Hiding poor metrics or limitations instead of explaining the next engineering step

Evidence required for GitHub or portfolio

- ☐ Stable release tag, deployment link, demo video, and professional case-study README
- ☐ Load/reliability/security reports and service-target note
- ☐ Real-user findings, fix log, before/after evidence, and retrospective

PART 1 · SHIP · COMMERCIAL AUTOMATION SPRINT

WEEK 10 — n8n + WhatsApp automation sprint and portfolio packaging

DONE WHEN A business owner can understand the prototype's value and limits, and a technical reviewer can reach every major project, metric, demo, and repository from one portfolio page.

Goal

Build one credible local-business automation prototype and package the engineering portfolio for real conversations.

Why this matters

WHY THIS MATTERS A focused automation sprint can create a short path to local SME validation or paid work while preserving the coded RAG/agent projects as the technical core.

What to learn and apply

- n8n workflow fundamentals, webhooks, test data, WhatsApp Business Platform test setup, and human handoff
- Business intent mapping, approved knowledge, privacy, failure handling, and operational ownership
- Portfolio positioning, project one-liners, architecture, evidence, demos, resume, and LinkedIn
- Commercial scoping without promising unsupported automation or production readiness

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Automation setup	• Run n8n locally or in a safe test environment and build a webhook → AI → response workflow. • Use a WhatsApp Business Platform test number or mock channel; do not connect a personal production account casually. • Add request IDs, basic logs, and a failure path before adding business logic.
Day 2	Local SME prototype	• Choose one business type such as tutoring center, clinic, restaurant, or service shop. • Support 3–4 narrow intents: hours, pricing, location, appointment/lead capture, and human handoff. • Answer only from approved business information and log unanswered questions for improvement.
Day 3	Test + pitch	• Test multilingual/unclear messages, missing data, repeated messages, opt-out, and handoff. • Create a one-page problem/solution scope, operating responsibilities, privacy note, and realistic price hypothesis. • Identify 5–10 relevant businesses through existing networks or local outreach.
Day 4	Portfolio structure	• Create a portfolio landing page or polished GitHub profile linking the assistant, RAG, agent, and automation work. • For each project show problem, users, architecture, stack, evaluation, security, cost, demo, deployment, and retrospective. • Remove unfinished repositories that dilute the story.

DAY	FOCUS	PRACTICAL TASKS
Day 5	Project packaging	- Standardize READMEs, diagrams, screenshots, demo videos, setup instructions, licenses, and sample environment files. - Verify every link from a logged-out browser/session when possible. - Create a concise portfolio index with the strongest project first.
Day 6	Career assets	- Update resume and LinkedIn around AI Application / AI Product Engineer outcomes, not a technology keyword dump. - Lead with 60+ mobile apps and 30+ live Play Store releases as proof of shipping ability. - Add quantified evaluation, latency/cost, deployment, and user-validation results from the AI projects.
REST	Scheduled recovery	- Full day off after the commercial and portfolio sprint. - Do not begin outreach while exhausted; quality and personalization matter.

Project gate

DELIVERABLE	MILESTONE
WhatsApp/n8n SME prototype + portfolio hub + updated resume and LinkedIn	A credible commercial prototype and a coherent public story connecting product-shipping experience with measured AI engineering work.

Testing requirements

- ☐ Intent, unknown-question, duplicate-message, handoff, and failure-path tests
- ☐ Portfolio link, deployment, setup, and logged-out-access checks
- ☐ Privacy and scope review before showing the prototype to a real business

Common mistakes

- Letting low-code automation replace the coded engineering spine
- Connecting real customer data before privacy, consent, ownership, and support are clear
- Listing tools without explaining product problems, decisions, or measured outcomes

Evidence required for GitHub or portfolio

- ☐ Workflow diagram, test report, demo video, scope/privacy note, and business pitch
- ☐ Portfolio hub, updated resume, and LinkedIn profile
- ☐ Verified links and a project-evidence checklist

PART 1 · SHIP · MARKET ENTRY

WEEK 11 — Freelance outreach, internships, and technical interview readiness

DONE WHEN Applications and proposals are live across targeted channels, follow-ups are scheduled, and project explanations are clear enough for both a hiring engineer and a business client.

Goal

Convert completed engineering evidence into targeted conversations with clients, internships, and early-career AI teams.

Why this matters

WHY THIS MATTERS A portfolio creates value only when the right people see it and Muhammad Zaid can explain the architecture, evaluation, trade-offs, and business outcome clearly.

What to learn and apply

- Role targeting, offer design, personalized proposals, local and global outreach, and follow-up tracking
- AI/RAG/agent system design explanations, evaluation methodology, cost/reliability trade-offs, and demos
- Internship applications, technical writing, feedback loops, and pipeline management
- Truthful positioning as a product shipper adding production AI capability

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Target + offers	• Create a role list: AI Application Engineer, LLM Application Engineer, AI Product Engineer, and AI Backend Engineer. • Define two narrow freelance offers based on shipped projects, with scope, exclusions, evidence, and starting price range. • Build a tracking sheet for lead, need, date, message, next action, status, and learning.

DAY	FOCUS	PRACTICAL TASKS
Day 2	Global outreach	- Send 5–8 personalized proposals that reference the buyer’s actual data, workflow, risk, and expected outcome. - Link only the closest project and one relevant metric. - Avoid bulk templates, unsupported promises, or fake production claims.
Day 3	Local outreach	- Contact 5–10 relevant Lahore/Pakistan businesses, startups, university contacts, or agencies with a concise demonstration. - Ask discovery questions before proposing a solution. - Offer a bounded pilot with clear data/privacy and support responsibilities.
Day 4	Internships + roles	- Apply to 5–10 well-matched internships or junior AI/backend roles with tailored summaries. - Emphasize product delivery, FastAPI bridge, evaluation, deployment, and honest learning scope. - Record the repeated skill gaps appearing in job descriptions.
Day 5	Interview practice	- Explain the RAG pipeline, agent state graph, security boundaries, provider strategy, and evaluation without slides. - Practice diagnosing a failed retrieval, runaway agent, high latency, and cost spike. - Prepare one architecture trade-off story and one real failure/learning story per major project.
Day 6	Technical writing	- Publish 2–3 concise posts: a retrieval failure and fix, an agent safety decision, and a product lesson. - Use concrete diagrams and metrics; avoid hype or generic AI claims. - Follow up on earlier conversations with one useful artifact.
REST	Scheduled recovery	- Final scheduled full rest day of the summer sprint. - No applications, outreach, or content publishing.

Project gate

DELIVERABLE	MILESTONE
Active opportunity pipeline with targeted applications, proposals, interview stories, and technical posts	The portfolio is in market, feedback is being collected, and Muhammad Zaid can defend the engineering decisions behind every major project.

Testing requirements

- ☐ Every proposal references a real requirement and one matching artifact
- ☐ Mock interview covers architecture, debugging, evaluation, security, cost, and limitations
- ☐ Tracking sheet includes next action and learning for every contact

Common mistakes

- Applying to every AI role without matching the problem, stack, or seniority
- Leading with tool names instead of product outcomes and evidence
- Stopping after one unanswered message or failing to record market feedback

Evidence required for GitHub or portfolio

- ☐ Redacted opportunity tracker and tailored application examples
- ☐ Recorded mock interview or written answer bank
- ☐ Published technical posts and response/feedback notes

PART 1 · SHIP · TRANSITION TO AI PRODUCT ENGINEERING

WEEK 12 — Delivery, retrospective, competency audit, and next-six-month plan

DONE WHEN The summer work is reproducible, documented, measurable, and mapped directly into the next six months of backend, data, SaaS, cloud, and Flutter product engineering.

Goal

Close the summer sprint professionally and convert its evidence into a precise continuation plan for backend, SaaS, cloud, and Flutter integration.

Why this matters

WHY THIS MATTERS Reflection and gap analysis prevent random learning. The next stage should be driven by demonstrated gaps between working AI demos and complete production products.

What to learn and apply

- Client/pilot delivery, documentation, testimonials, portfolio maintenance, and ethical handoff
- Competency audit across AI, backend, data, security, deployment, product, and mobile integration
- Technical retrospective, evidence inventory, semester capacity planning, and prioritization
- Transition from 12-week AI foundation to the 24-week AI Product Engineer expansion

Execution plan

DAY	FOCUS	PRACTICAL TASKS
Day 1	Deliver or continue	- Complete any landed pilot with written scope, progress updates, acceptance checks, handoff, and support limits. - If no work landed, continue targeted outreach without treating timing as technical failure. - Request a testimonial only after a real outcome is delivered.
Day 2	Repository audit	- Run every setup guide from a clean environment and repair broken dependencies, scripts, links, or sample configs. - Archive experiments that are not portfolio-ready. - Add issue lists for known limitations and next production steps.
Day 3	Evidence inventory	- Create a table for each project: architecture, code, tests, eval, security, cost, deployment, demo, user feedback, retrospective. - Close missing critical evidence or mark it explicitly as incomplete. - Capture final screenshots and stable release tags.
Day 4	Competency audit	- Score each final checklist item as proven, partial, or not yet proven and link the evidence. - Identify the three highest-risk gaps: likely professional backend design, persistent data/jobs, and cloud operations. - Use those gaps to sequence Part 2 rather than adding random technologies.
Day 5	Retrospective + plan	- Write what was built, what failed, which metrics improved, what users valued, and what remains fragile. - Block 15–20 hours/week for the continuation roadmap and choose a realistic start date. - Define a stop-doing list to protect university workload and quality.
Day 6	Disconnect	- Take a genuine recovery day before beginning the continuation roadmap. - Do not disguise new work as ‘light review.’
Day 7	Transition	- Read the Part 2 phase gates and prepare only the FastAPI backend phase environment. - Keep the strongest AI project as the product that will be professionalized across the next six months. - Publish a concise summer summary and next-stage commitment.

Project gate

DELIVERABLE	MILESTONE
Summer engineering retrospective + evidence inventory + signed 24-week continuation plan	The 12-week sprint ends with stable releases, honest competency evidence, an active opportunity pipeline, and a prioritized path to full AI product engineering.

Testing requirements

- ☐ Clean-environment repository/setup checks and final deployment smoke tests
- ☐ Evidence audit for every portfolio requirement and competency item
- ☐ Continuation schedule checked against university and client commitments

Common mistakes

- Beginning new frameworks before closing documentation, tests, and evidence gaps
- Scoring competencies by familiarity rather than a working artifact
- Assuming project completion automatically equals senior-level engineering capability

Evidence required for GitHub or portfolio

- ☐ Final release tags, deployment checks, and portfolio evidence inventory
- ☐ Written retrospective and proven/partial/not-proven competency audit
- ☐ 24-week calendar with phase gates and a protected weekly time budget

PART 2 · POST-SUMMER EXPANSION

PART 2 — 24-WEEK AI PRODUCT ENGINEER EXPANSION

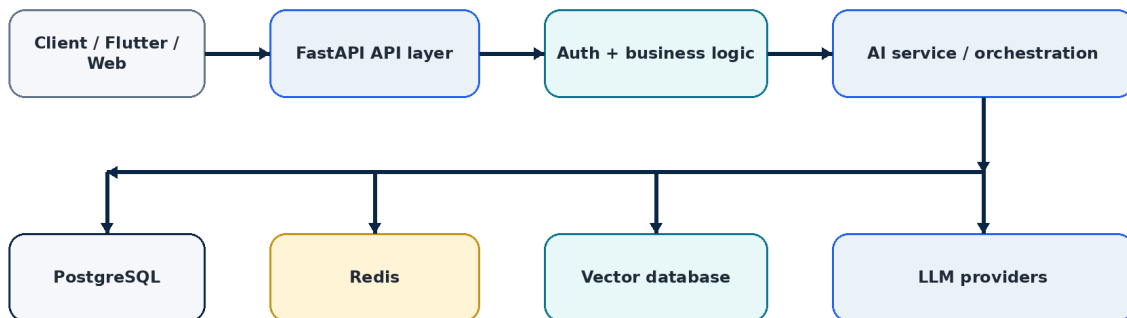
From AI applications to complete AI products

TRANSITION RULE Part 1 proves the AI application layer. Part 2 turns the strongest project into a multi-user, data-backed, cloud-operated product. Flutter remains deliberately last because it is already Muhammad Zaid's strength.

Recommended pace: 24 weeks at approximately 15–20 focused hours per week alongside university. If university or client commitments reduce capacity, preserve the phase order and extend the calendar. Do not sacrifice testing, evaluation, security, documentation, or health to maintain an arbitrary deadline.

EXECUTION RULE When behind, reduce optional scope before reducing testing, evaluation, security, documentation, or rest.

Production AI backend reference architecture



Datastore responsibilities — use each system deliberately

COMPONENT	AUTHORITATIVE RESPONSIBILITY	DO NOT USE IT AS
PostgreSQL	Users, organizations, roles, permissions, product data, document metadata, conversations, feedback, usage events, and billing ledger	A semantic search engine or disposable cache
Redis	Cache, session/revocation state, rate counters, queue coordination, short-lived locks, and temporary workflow state	The durable source of truth for accounts, permissions, or billing

COMPONENT	AUTHORITATIVE RESPONSIBILITY	DO NOT USE IT AS
Vector database	Embeddings, chunk metadata required for retrieval, similarity search, filtering, and index lifecycle	The authority for user identity, permission policy, or original files
Object storage	Original uploads, derived artifacts, exports, and durable file versions	A relational database or permission decision engine
LLM provider	Generation, structured extraction, classification, and bounded tool decisions through an application-controlled interface	A database, permission system, or keeper of privileged business rules
FastAPI services	Authentication boundary, authorization, business rules, orchestration, stable contracts, errors, and audit events	A single route that directly calls every external dependency

Milestone gates

GATE	REQUIRED PROOF	ADVANCE WHEN
A · AI application	Structured outputs, provider switching, streaming, usage telemetry, tests	Week 1 evidence is reproducible
B · RAG	Multi-document ingestion, retrieval metrics, cited/no-answer behavior, security	Week 5 deployed evaluation passes
C · Agent	Bounded tools, trajectory evaluation, authorization, recovery, handoff	Week 7 deployed release passes
D · Backend + data	Versioned API, auth, PostgreSQL, Redis, durable jobs, tenant isolation	Phases 1–2 tests pass
E · SaaS + cloud	Multi-user value loop, limits/cost, CI/CD, monitoring, backup/rollback	Phases 3–4 release gates pass
F · Mobile product	Flutter client, streaming, files, agent confirmations, failure UX	Phase 5 external test passes
G · AI Product Engineer	Flagship pilot, evaluations, security, operations, full portfolio evidence	Phase 6 is deployed and used

PART 2 · WEEKS 13–16

PHASE 1 — Professional FastAPI AI Backend Engineering

DONE WHEN A new client can authenticate, call a versioned API, stream AI output, upload files, receive stable errors, and use complete documentation while routes remain thin and business logic remains testable.

Goal

Upgrade FastAPI from a thin model wrapper into a tested, versioned, secure backend that can serve multiple clients and AI workflows.

Why this matters

WHY THIS MATTERS The backend owns product rules, identity, authorization, error contracts, orchestration boundaries, and operational behavior. It must remain maintainable as models and interfaces change.

Implementation decision

IMPLEMENTATION DECISION Primary path: Python + FastAPI + REST/SSE + Pydantic/SQLAlchemy/Alembic.

Reason: This path aligns typed Python AI services with explicit API contracts, streaming, persistence, migrations, and professional testing.

Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Approve ADRs for API style, authentication approach, ORM boundaries, and streaming before the phase gate.

What to learn and apply

- Project structure: app entrypoint, routers, schemas, services, repositories, domain models, configuration, and dependencies
- REST design, API versioning, validation, stable errors, pagination, idempotency, OpenAPI documentation, and middleware
- JWT access/refresh flows, password handling through trusted libraries, roles, permissions, and authorization tests
- Async I/O, streaming/SSE, WebSockets where justified, uploads, cancellation, rate limiting, unit tests, and integration tests

Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 13	Architecture and contracts	- Refactor one summer project into routers → services → repositories/providers with configuration isolated under a core layer. - Define `v1` schemas, error envelope, request/trace IDs, dependency injection, and startup validation. - Keep LLM/RAG/agent code behind interfaces so routes never know provider details.
Week 14	Identity and authorization	- Implement user registration/login, secure password hashing, access/refresh tokens, revocation strategy, and role/permission checks. - Build an authorization matrix and test every protected endpoint as anonymous, owner, member, and admin. - Do not invent authentication cryptography or trust claims sent by the client.
Week 15	Async product behavior	- Implement streaming responses, upload limits, request cancellation, pagination, middleware, CORS policy, and rate limiting. - Use FastAPI BackgroundTasks only for small post-response work; route long ingestion to the worker architecture in Phase 2. - Load-test I/O paths and remove blocking calls from async routes.
Week 16	Testing and API release	- Add unit tests for services and integration tests for API/database/provider boundaries. - Publish OpenAPI examples, error codes, authentication flow, local setup, and a versioning/deprecation policy. - Release a client-ready backend that works through curl/API docs without a custom UI.

Phase gate

DELIVERABLE	MILESTONE
Production AI Backend API with authentication, authorization, streaming, uploads, versioning, documentation, and tests	A maintainable backend boundary that can power Streamlit today and Flutter later without duplicating business or AI logic.

Testing requirements

- ☐ Service unit tests plus API/database/provider integration tests on critical paths
- ☐ Authorization matrix, token expiry/refresh/revocation, rate-limit, upload, pagination, and error-contract tests
- ☐ Concurrent streaming/cancellation check, OpenAPI validation, and clean-environment startup test

Common mistakes

- Creating a folder hierarchy without real separation of responsibilities
- Mixing sync blocking libraries inside async routes or using in-process tasks for durable work
- Checking authorization only in the UI or trusting role/user IDs supplied by the client

Evidence required for GitHub or portfolio

- ☐ Backend architecture diagram and directory map
- ☐ OpenAPI specification, auth flow, authorization matrix, and test report
- ☐ Deployment/demo through API docs plus refactoring retrospective

PART 2 · WEEKS 17–20

PHASE 2 — PostgreSQL, Redis, Background Jobs, and AI Data Pipelines

DONE WHEN A file upload returns quickly, processing continues durably, progress survives restarts, failures are visible and retryable, and no user can read or retrieve another workspace's data.

Goal

Build the durable data and asynchronous processing platform required by multi-user AI products.

Why this matters

WHY THIS MATTERS Real products must preserve users, permissions, documents, conversations, usage, and job state reliably while processing large files outside request/response time limits.

Implementation decision

IMPLEMENTATION DECISION Primary path: PostgreSQL + Redis + one durable worker stack + one object store + pgvector or Qdrant.

Reason: Each datastore has one authoritative responsibility while long-running ingestion survives request and worker failures.

Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Approve ADRs for worker/broker, vector storage, object storage, retention, and tenant-scoping before the phase gate.

What to learn and apply

- PostgreSQL schema design, relationships, constraints, transactions, indexes, query plans, SQLAlchemy, and Alembic migrations
- Redis for cache, sessions/revocation, rate counters, job coordination, and temporary state—not primary truth
- Object storage for original files and derived artifacts; vector database for embeddings and semantic retrieval
- Idempotent ingestion jobs, queues/workers, retries, dead-letter handling, status, cancellation, metadata, validation, and data quality

Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 17	Relational product model	• Model users, organizations/workspaces, memberships, roles, projects, documents, conversations, messages, feedback, usage events, and billing ledger entries. • Add primary/foreign keys, unique/check constraints, timestamps, tenant ownership, and transaction boundaries. • Create a migration from an empty database and seed only non-sensitive development data.
Week 18	Queries, indexes, migrations	• Implement repository queries with SQLAlchemy and profile representative list/search/usage queries. • Add indexes based on measured query patterns; inspect query plans before and after. • Practice forward migration, rollback in a test environment, and backup/restore verification.
Week 19	Redis and durable jobs	• Add Redis-backed rate limits, cache keys scoped by tenant/permissions, token/session revocation, and temporary task state. • Introduce a worker queue for document processing with idempotency key, retry policy, timeout, cancellation, and failure status. • Ensure PostgreSQL remains the system of record and jobs can recover after worker restart.
Week 20	Reusable ingestion platform	• Connect object storage, PostgreSQL metadata, worker processing, vector indexing, progress events, and cleanup. • Validate schema, required metadata, duplicate content, parser quality, and tenant access before indexing. • Expose upload → accepted → processing → ready/failed lifecycle through the API and run chaos/restart tests.

Phase gate

DELIVERABLE	MILESTONE
Reusable asynchronous AI ingestion platform backed by PostgreSQL, Redis, object storage, and a vector database	Durable, restart-safe document processing with explicit ownership, lifecycle state, data quality checks, and observable failures.

Testing requirements

- ☐ Migration from zero, migration upgrade/rollback in test, constraint, transaction, and backup/restore checks
- ☐ Tenant isolation, cache-key isolation, rate-limit, session-revocation, and stale-cache tests
- ☐ Idempotent duplicate jobs, worker restart, retry exhaustion, cancellation, corrupt document, and cleanup tests

Common mistakes

- Using Redis or the vector store as the source of truth for user/product records
- Adding indexes by intuition or running destructive migrations without backup and rollback planning
- Making job retries create duplicate chunks, usage events, or external actions

Evidence required for GitHub or portfolio

- ☐ Entity-relationship diagram and datastore responsibility matrix
- ☐ Alembic history, query-plan comparison, worker lifecycle diagram, and test report
- ☐ Failure/recovery demo and ingestion data-quality dashboard or summary

PART 2 · WEEKS 21–24

PHASE 3 — AI Product Architecture and SaaS Engineering

DONE WHEN Multiple users can use the product safely, limits and costs are attributable, onboarding reaches a defined first-value event, and beta feedback drives a documented product decision.

Goal

Transform the backend and AI capabilities into a coherent multi-user SaaS product with measurable activation, usage, feedback, and unit economics.

Why this matters

WHY THIS MATTERS AI Product Engineers connect technical capability to user value. Accounts, permissions, limits, onboarding, analytics, feedback, and cost controls determine whether the system can become a product.

Implementation decision

IMPLEMENTATION DECISION Primary path: FastAPI SaaS services + deterministic authorization + PostgreSQL usage/entitlement ledger.

Reason: This keeps identity, access, product rules, limits, feedback, analytics, and cost controls outside the model.

Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Approve ADRs for custom versus managed identity, tenancy, entitlements, and mock-versus-real billing before the phase gate.

What to learn and apply

- Problem selection, user interviews, jobs-to-be-done, requirements, MVP scope, non-goals, and acceptance metrics
- Multi-user/workspace architecture, roles, usage limits, admin controls, file storage, onboarding, and product lifecycle
- Subscription/billing concepts, entitlement checks, usage ledger, quotas, AI cost allocation, and auditability
- Product analytics, activation funnel, task success, feedback, retention signals, support, and model/provider routing

Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 21	Problem and MVP	• Interview or observe 3–5 target users and select one painful repeated workflow with data access you can obtain legally. • Write a product requirements document: user, job, workflow, risks, MVP, non-goals, success/guardrail metrics, and launch criteria. • Prefer a narrow product that completes one task over a broad ‘AI assistant for everything.’
Week 22	Multi-user product core	• Implement workspaces, membership, roles, invitations, ownership, document/project boundaries, and an admin view/API. • Add onboarding that leads a new user to first successful AI outcome with sample or permitted data. • Test tenant isolation across PostgreSQL, storage, vector retrieval, cache, traces, and exports.

WEEK	FOCUS	PRACTICAL TASKS
Week 23	Entitlements and economics	- Implement plan/entitlement concepts, usage counters, limits, over-limit behavior, and an append-only usage/billing ledger. - Track model/provider, tokens, storage, jobs, successful tasks, and estimated cost per user/workspace. - Use a billing sandbox or mock entitlement service; the goal is architecture and auditability, not payment-provider expertise.
Week 24	Analytics, feedback, and beta	- Instrument signup → onboarding → first document/task → first successful answer/action → return usage. - Collect explicit helpful/not-helpful feedback plus reason and trace ID; connect it to evaluation triage. - Run a small beta with 3–5 users and prioritize fixes by user value, severity, and frequency.

Phase gate

DELIVERABLE	MILESTONE
Multi-user AI SaaS backend and beta product with onboarding, entitlements, usage/cost tracking, analytics, feedback, and admin controls	A real product loop: users onboard, complete a valuable AI task, provide feedback, consume controlled resources, and generate evidence for the next iteration.

Testing requirements

- ☐ End-to-end signup/onboarding/first-success flow and role/permission matrix
- ☐ Cross-tenant isolation across relational data, object storage, vector retrieval, cache, logs, and exports
- ☐ Usage-limit, entitlement, ledger reconciliation, feedback linkage, and admin-action tests

Common mistakes

- Building billing before validating the user problem or confusing subscriptions with product-market fit
- Using page views as the main metric instead of successful user tasks and return behavior
- Treating workspace filtering as an optional UI concern rather than an end-to-end security boundary

Evidence required for GitHub or portfolio

- ☐ Product requirements document, user-flow diagram, and SaaS architecture
- ☐ Entitlement/usage ledger design, analytics event dictionary, and unit-cost worksheet
- ☐ Beta findings, product changes, screenshots/demo, and test report

PART 2 · WEEKS 25–28

PHASE 4 — Cloud, DevOps, Reliability, and Production Operations

DONE WHEN A commit can move through automated quality gates to staging and controlled production, the team can detect and diagnose failure, and data/release recovery has been practiced—not assumed.

Goal

Deploy the SaaS product through a repeatable pipeline and operate it with secure configuration, monitoring, backups, health checks, and basic scaling.

Why this matters

WHY THIS MATTERS Shipping is not the final command. Production engineering ensures releases are reproducible, failures are visible, data is recoverable, and cost/performance remain controlled.

Implementation decision

IMPLEMENTATION DECISION Primary path: Docker/Compose + GitHub Actions + one cloud + structured logs/trace IDs + one AI tracer.

Reason: One complete delivery and operations path builds reproducible deployment, monitoring, recovery, and rollback skill.

Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Approve ADRs for cloud, compute, managed data, observability, backup/restore, and the small Terraform/OpenTofu staging exercise before release.

What to learn and apply

- Dockerfile, images, multi-stage builds, non-root runtime, Docker Compose, environment configuration, and secrets
- Linux server basics, cloud compute, object storage, managed PostgreSQL, Redis, DNS/TLS, networking, CORS, and environment separation
- GitHub Actions for lint/test/build/scan/migrate/deploy, deployment approvals, rollback, and release notes

- Structured logs, traces, metrics, errors, health/readiness, backups, restore drills, alerts, incident notes, scaling, and cost

Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 25	Containers and local stack	• Create a production-oriented Dockerfile and Compose stack for API, worker, PostgreSQL, Redis, and supporting services. • Run as non-root, pin dependencies, minimize image size, add health checks, and keep secrets out of images. • Reproduce the full system from a clean machine/environment using documented commands.
Week 26	Cloud foundation	• Choose one cloud path and deploy compute, managed PostgreSQL, object storage, Redis as appropriate, DNS/TLS, and private/public network boundaries. • Create separate development/staging/production configuration and least-privilege service credentials. • Document data region, retention, backup, and provider cost assumptions.
Week 27	CI/CD	• Build a GitHub Actions pipeline: lint/type/test → image build → security/dependency checks → staging deploy → migration → smoke test → production approval. • Use immutable image tags and record release metadata; practice rollback to the prior release. • Prevent failed migrations or tests from reaching production.
Week 28	Operate and scale	• Create dashboards/alerts for availability, errors, p95 latency, queue depth, job failures, token/cost, and AI task success. • Run dependency outage, worker restart, disk/storage issue, and backup-restore drills; write an incident note. • Document basic horizontal scaling and caching; keep Kubernetes as a future topic until real scale demands it.

Phase gate

DELIVERABLE	MILESTONE
Cloud-deployed AI SaaS with containerized services, CI/CD, observability, backups, release/rollback, and operating runbook	A repeatable path from commit to monitored production with recovery evidence and transparent operating cost.

Testing requirements

- ☐ Clean Docker/Compose build, non-root runtime, secret scan, health/readiness, and startup-order checks
- ☐ CI failure gates, staging smoke test, migration safety, immutable release, and rollback drill
- ☐ Alert test, dependency outage, worker restart, backup/restore, and basic load/scaling exercise

Common mistakes

- Treating Docker Compose as production orchestration without understanding its limits
- Storing secrets in repository variables, images, logs, or client applications
- Adding Kubernetes before mastering containers, networking, monitoring, backups, and actual scaling needs

Evidence required for GitHub or portfolio

- ☐ Docker/Compose files, cloud architecture, environment map, and threat notes
- ☐ CI/CD workflow, release/rollback record, monitoring screenshots, and operating runbook
- ☐ Backup/restore and incident drill evidence plus monthly cost estimate

PART 2 · WEEKS 29–31

PHASE 5 — Flutter AI Product Integration — Final Integration Phase Only

DONE WHEN The Flutter app can be installed by an external tester who can authenticate, upload, stream, review citations, confirm agent actions, recover from failures, and understand usage limits.

Goal

Use Muhammad Zaid’s strongest existing capability to turn the production backend into a polished mobile AI product.

Why this matters

WHY THIS MATTERS Flutter is a product advantage after the AI/backend system exists. The mobile application should consume stable APIs, handle uncertainty and failure well, and never contain privileged AI or data logic.

Implementation decision

IMPLEMENTATION DECISION Primary path: Flutter as a protected client of the production FastAPI platform.

Reason: Flutter compounds existing product strength without moving secrets, authorization, privileged tools, or provider access into the client.

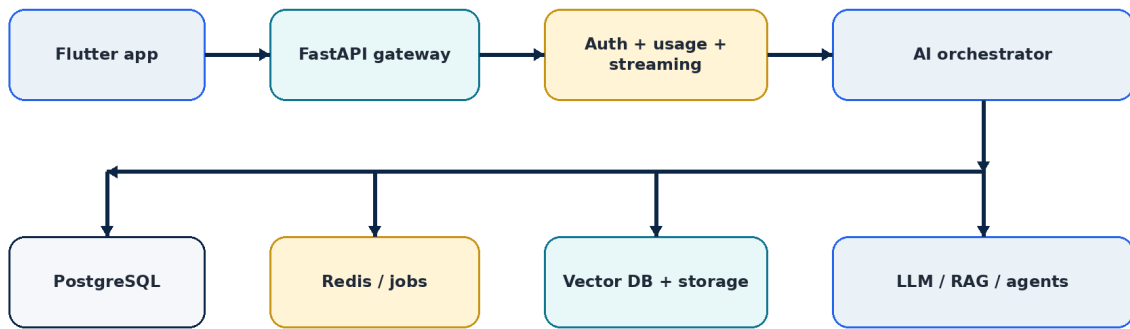
Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Approve ADRs for REST/SSE/WebSocket use, secure token storage, file transfer, offline/error UX, and mobile distribution before the phase gate.

What to learn and apply

- Typed API client, authentication/refresh, secure token storage, streaming/SSE or WebSockets, cancellation, and retries
- Conversation history, citations, file upload/progress, job status, agent confirmations, human handoff, and error states
- Usage limits, feedback, analytics, accessibility, responsive UX, offline/poor-network behavior, and privacy
- Clean Architecture integration without duplicating backend authorization, prompts, keys, or business rules in Flutter

Final integration architecture: Flutter as a product advantage



Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 29	Client foundation	- Generate or implement a typed client for the versioned API with auth refresh, request/trace IDs, stable error mapping, and environment configuration. - Implement streaming tokens/events with cancellation and reconnection behavior. - Keep provider keys and privileged prompts exclusively on the backend.
Week 30	AI product flows	- Build onboarding, workspace/project selection, conversation history, file upload/progress, cited answers, and feedback. - Add agent action preview/confirmation, tool progress, safe failure, and human handoff UI. - Represent no-answer and conflicting-source states honestly rather than as generic chat errors.
Week 31	Mobile quality gate	- Test expired tokens, slow/offline network, stream interruption, duplicate submission, large upload, over-limit, job failure, and server rollback. - Add accessibility, analytics, privacy disclosure, crash/error reporting, and a clean release build. - Run usability sessions and fix the highest-impact mobile friction before the capstone.

Phase gate

DELIVERABLE	MILESTONE
Production-connected Flutter AI client demonstrating streaming, auth, history, files, RAG, agent tools, limits, and resilient UX	The existing Flutter strength now multiplies a production AI platform instead of masking a demo-only backend.

Testing requirements

- ☐ Auth refresh/revocation, typed error mapping, streaming interruption/cancellation, retry, and duplicate-submit tests
- ☐ File upload/progress/failure, no-answer/citation, confirmation/handoff, and usage-limit flows
- ☐ Accessibility, poor-network, privacy, analytics, and release-build smoke tests

Common mistakes

- Putting LLM keys, system prompts, authorization decisions, or business rules in the mobile app
- Showing token streams without cancellation, error recovery, or persistence
- Treating AI uncertainty as a generic loading or network error

Evidence required for GitHub or portfolio

- ☐ Flutter/backend architecture diagram and typed API contract
- ☐ Mobile test report covering failure states and security boundaries
- ☐ Installable build, demo video, usability findings, and store-ready screenshots

PART 2 · WEEKS 32–36

PHASE 6 — Flagship AI SaaS + Mobile Product

DONE WHEN The product is deployed, used by external testers, evaluated against versioned datasets, protected by real access controls, observable in production, costed per successful task, and fully documented for a technical reviewer.

Goal

Ship one complete AI Document Intelligence & Business Assistant product that proves end-to-end AI Product Engineer capability.

Why this matters

WHY THIS MATTERS This capstone integrates product discovery, RAG, agents, FastAPI, PostgreSQL, Redis, asynchronous jobs, SaaS controls, cloud operations, and Flutter into one coherent real-user outcome.

Implementation decision

IMPLEMENTATION DECISION Primary path: One consolidated production stack selected from measured product requirements.

Reason: The capstone proves deliberate engineering choices, not framework collection, through real users, release evidence, and operations.

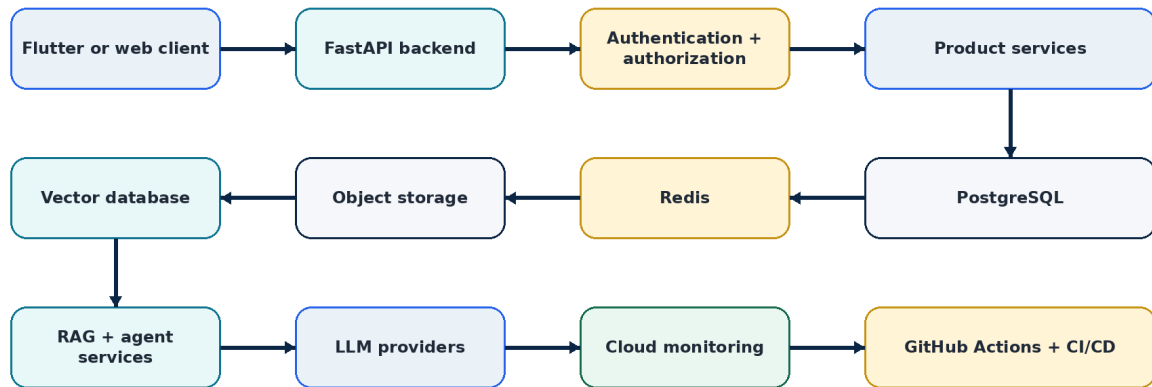
Full alternatives: see Technology Decision Guide — Primary Paths, Alternatives, and Trade-Offs.

Decision checkpoint: Freeze an ADR index for every primary technology, alternatives considered, trade-offs, risks, and revisit triggers before pilot launch.

What to learn and apply

- Flagship scope: workspace-based document intelligence plus bounded business-assistant actions
- Cited Q&A, structured summaries/action items, approved agent tools, human confirmation/handoff, and audit trail
- Accounts, roles, usage limits, analytics, feedback, cost tracking, admin, onboarding, and production deployment
- Evaluation, security, reliability, pilot users, case study, demo, documentation, and launch retrospective

Final AI SaaS architecture



Four-to-five-week execution plan

WEEK	FOCUS	PRACTICAL TASKS
Week 32	Product and release design	• Select one target segment and one repeated workflow; write PRD, user journeys, architecture, threat model, data policy, golden dataset, launch metrics, and non-goals. • Define the smallest flagship: upload business documents, ask cited questions, extract actions, and run one confirmed business tool. • Freeze requirements before the build and create a release board tied to acceptance tests.
Week 33	Knowledge system	• Implement asynchronous multi-format ingestion, metadata/permissions, hybrid retrieval, reranking, cited/no-answer generation, and retrieval debug tools. • Run the golden RAG evaluation and fix the largest failure cluster. • Instrument latency, tokens, cost, and feedback linkage.
Week 34	Agent + SaaS core	• Add the bounded LangGraph workflow, narrow tools, authorization, confirmation, max-step, retry, and human handoff. • Complete workspaces, roles, usage/entitlements, usage ledger, admin controls, audit events, and background job visibility. • Run trajectory, security, tenant-isolation, and failure-injection suites.
Week 35	Flutter experience	• Connect onboarding, auth, workspaces, upload/progress, streaming conversation, citations, agent action confirmation, history, limits, and feedback. • Polish offline/slow/error states and accessibility; keep all privileged logic on the backend. • Run end-to-end mobile tests against staging and fix release blockers.

WEEK	FOCUS	PRACTICAL TASKS
Week 36	Production pilot + case study	• Deploy production, verify monitoring/alerts/backups/rollback, and run full evaluation on the release build. • Pilot with 3–5 real users or one small business; collect task success, time saved, feedback, failures, retention intent, and cost. • Publish repository, README, architecture, API docs, test/eval reports, security/cost notes, demo video, deployment link, and retrospective.

Phase gate

DELIVERABLE	MILESTONE
Flagship AI Document Intelligence & Business Assistant — cloud SaaS backend + Flutter application	A real user can move from onboarding to a trustworthy AI outcome and a confirmed agent action in a deployed, observable, multi-user product.

Testing requirements

- ☐ Full RAG, agent trajectory, auth/authorization, tenant isolation, prompt-injection, data leakage, and output-validation suites
- ☐ Background job, provider outage, timeout/retry, cost ceiling, rate limit, load, backup/restore, deployment, and rollback checks
- ☐ Mobile end-to-end, usability, accessibility, analytics, and external pilot acceptance results

Common mistakes

- Expanding to multiple business workflows before one end-to-end outcome is reliable
- Calling the product production-ready without pilot, failure, security, recovery, and operating evidence
- Using polished Flutter screens to hide weak retrieval, unsafe tools, or incomplete backend architecture

Evidence required for GitHub or portfolio

- ☐ Live deployment and installable mobile build with complete demo
- ☐ Professional repository, architecture, API docs, test suite, evaluation report, security notes, cost analysis, and runbook
- ☐ Pilot outcomes, feedback-linked fixes, before/after metrics, and technical/product retrospective

PRODUCTION ENGINEERING STANDARD

Production Engineering Standard

Release gates for every serious AI product

RELEASE PRINCIPLE A product is ready when quality, security, reliability, performance, cost, and operational recovery meet explicit evidence-based thresholds. A polished demo is not a release gate.

Reference API surface

AREA	REPRESENTATIVE ENDPOINTS	OWNERSHIP
Identity	`/v1/auth/*`, `/v1/users/me`, `/v1/workspaces/*`	Authentication, membership, roles, invitations, profile
Documents	`/v1/documents`, `/v1/documents/{id}`, `/v1/jobs/{id}`	Upload, status, list, delete, processing lifecycle
AI interaction	`/v1/chat`, `/v1/chat/stream`, `/v1/agent/runs/*`	Cited answers, streaming events, agent confirmation/cancel/status
Product	`/v1/usage`, `/v1/feedback`, `/v1/admin/*`	Limits, usage/cost, feedback, admin controls
Operations	`/health/live`, `/health/ready`, internal metrics/traces	Liveness, dependency readiness, monitoring—not sensitive internals

Evaluation and release matrix

DIMENSION	MEASURE	STARTING RELEASE GATE
Retrieval	Recall/hit rate at k, ranking quality, filter correctness, failure categories	Project-defined threshold on a versioned representative dataset; no regression from approved baseline
Answers	Groundedness, relevance, completeness, citation correctness, no-answer behavior	Material claims trace to evidence; critical citation/access failures are zero
Agents	Task success, tool choice/order, arguments, prohibited actions, steps, handoffs	No unauthorized action; bounded steps; target success met on versioned trajectories
Security	Prompt injection, cross-tenant access, data leakage, output validation, secret exposure	Zero unresolved critical/high access-control or data-leakage findings
Reliability	Timeout/fallback, provider/tool failure, job restart, cancellation, idempotency	Known failures recover, hand off, or fail transparently without corrupting state

DIMENSION	MEASURE	STARTING RELEASE GATE
Performance	p50/p95 latency, queue time, concurrent errors, cache behavior	Product-specific target met under representative load; bottlenecks documented
Cost	Tokens, model/provider, storage/jobs, cost per query and successful task	Per-user/workspace budget enforced; cost regression explained and approved
Operations	Health, logs, traces, alerts, backup/restore, rollback	Detection and recovery drills pass; operator runbook is current

Security and privacy checklist

- ☐ Authenticate every non-public request and authorize each resource/tool action in application code.
- ☐ Scope PostgreSQL queries, object keys, cache keys, vector filters, traces, and exports by tenant/workspace.
- ☐ Treat user text, retrieved documents, tool output, and web content as untrusted data—not system instructions.
- ☐ Validate tool arguments and model outputs; require confirmation for material external actions.
- ☐ Limit upload types/sizes, scan or quarantine where appropriate, and isolate parsing failures.
- ☐ Store secrets only in approved server-side configuration; rotate and revoke; never ship provider keys in Flutter.
- ☐ Minimize logs and traces, redact sensitive fields, define retention/deletion, and document provider data handling.
- ☐ Run prompt-injection, cross-tenant, data-leakage, authorization, and export tests before every material release.

Reliability patterns

FAILURE	ENGINEERING RESPONSE	USER/OPERATOR OUTCOME
Provider timeout/rate limit	Deadline, bounded retry with jitter, fallback if compatible, circuit/budget control	Transparent delay/failure; trace identifies provider and attempt
Bad or missing retrieval	Debug evidence, rewrite once, threshold/no-answer, human feedback path	No fabricated answer; sources and uncertainty visible
Tool failure	Typed error, retry only if safe, idempotency, compensation/dry-run, handoff	No duplicate side effect; actionable status
Long document job	Durable queue, status/progress, timeout, retry, cancellation, dead-letter review	Request returns quickly; job survives worker restart
Deployment regression	Automated gates, staging smoke, immutable release, migration safety, rollback	Prior stable release restored; incident recorded

Definition of done for every major project

- ☐ A real user/business problem, target user, scope, non-goals, and acceptance criteria are written.
- ☐ Architecture diagram, ADR index, datastore responsibilities, API contract, threat/security notes, alternatives considered, trade-offs, and known limitations are documented.
- ☐ Unit/integration/end-to-end tests and a versioned evaluation dataset run from documented commands.
- ☐ Before/after metrics include quality, latency, reliability, and cost—not a single cherry-picked score.
- ☐ Deployment, logs/traces, health checks, failure handling, and recovery/rollback evidence exist.
- ☐ README, API docs, demo video, screenshots, deployment link, cost analysis, security notes, and retrospective are public or safely shareable.

FINAL PORTFOLIO

Final Portfolio

Five projects that prove the engineer profile

PORTFOLIO ANSWER After completing the required gates, can Muhammad Zaid build and ship real AI SaaS products? YES—because the portfolio proves the full path from model integration through data, backend, SaaS controls, cloud operations, Flutter, evaluation, security, and real users.

PROJECT	WHAT IT PROVES	MINIMUM ACCEPTANCE EVIDENCE
1 · LLM Assistant and Structured Extractor	Provider integration, prompting, structured output, streaming, reliability, usage telemetry	Two providers; validated schemas; golden tests; prompt-injection probes; README; demo; cost/latency summary
2 · Production RAG Knowledge System	Document ingestion, metadata, hybrid retrieval, citations, evaluation, security, deployment	Multi-format corpus; eval report; before/after metrics; no-answer/citation/access tests; API docs; live demo
3 · Tool-Using LangGraph Agent	Stateful workflows, tools, permissions, recovery, human handoff, trajectory evaluation	Tool contracts; graph; traces; trajectory dataset; denied-action and failure tests; deployed API; runbook
4 · Complete AI SaaS Product	Multi-user backend, PostgreSQL/Redis/jobs, roles, limits, analytics, cost, cloud operations	Onboarding; tenant isolation; usage ledger; CI/CD; monitoring; backup/rollback; external beta; product metrics
5 · Flagship Flutter AI Product	Mobile product advantage integrated with production AI backend	Auth; streaming; history; upload; RAG; agent confirmation; limits; error UX; deployed backend; pilot results

Required evidence package for every flagship project

- ☐ GitHub repository with a clean, reviewable history and release tag
- ☐ Professional README with problem, users, scope, features, architecture, and proof summary
- ☐ Architecture diagram and an indexed set of Architecture Decision Records
- ☐ Primary technology choices, alternatives considered, reasons for selection, trade-offs, risks, and revisit triggers
- ☐ Reproducible setup instructions and documented configuration contract
- ☐ API documentation with representative requests, responses, errors, and authorization behavior
- ☐ Automated unit, integration, and end-to-end tests appropriate to the project
- ☐ Versioned evaluation dataset with representative, adversarial, and failure cases
- ☐ Evaluation report with methodology, thresholds, results, and failure analysis
- ☐ Before-and-after quality, latency, reliability, and cost metrics
- ☐ Security test results covering authorization, injection, data leakage, secrets, unsafe tools, and residual risks
- ☐ Privacy notes covering data sources, provider handling, logging, retention, and deletion
- ☐ Cost analysis covering model, storage, jobs, and cost per query or successful task
- ☐ Latency analysis covering p50/p95 behavior, queue time, bottlenecks, and the selected product target
- ☐ Product screenshots that show the core workflow and failure states
- ☐ Concise demo video that shows the real product, evaluation evidence, and trade-offs
- ☐ Stable deployment link where safe and affordable
- ☐ User feedback or pilot evidence tied to product decisions
- ☐ Technical retrospective covering decisions, incidents, lessons, and next actions
- ☐ Known limitations and prioritized future improvements

Portfolio narrative

Lead with the strongest differentiator: Muhammad Zaid already knows how to ship mobile products. The portfolio should show that this product-shipping discipline now extends through professional AI systems and backend/cloud ownership. Present the flagship first, then use the other projects as focused evidence of individual capabilities.

FINAL COMPETENCY AUDIT

Competency Audit

AI Product Engineer proof checklist

STATUS RULE Use only three statuses: Proven, Partial, or Not Yet Proven. A competency can be marked Proven only when the required evidence exists, is linked, and can be reproduced. Familiarity, tutorial completion, or a partial notebook is Partial—not proof.

AI Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Select and integrate models through a provider-neutral boundary	Not Yet Proven	Provider adapter, two smoke tests, measured primary/fallback ADR	Python provider adapter	Hosted providers; local Ollama/vLLM	Link measured provider decision
Build reliable structured output and tool-calling behavior	Not Yet Proven	Golden dataset, schema validity, retry/failure results, cost/latency report	Pydantic + hosted LLM APIs	Provider-native schemas; custom parsers	Meet release threshold

LLM Application Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build prompt, context, streaming, and usage-aware product features	Not Yet Proven	Running assistant/extractor, REST/SSE contract, telemetry, tests	FastAPI + REST/SSE	WebSockets; polling; alternative backends	Link deployed feature evidence
Handle provider failures without corrupting product state	Not Yet Proven	Timeout, rate-limit, fallback, budget, and user-error tests	Application service layer	Circuit breakers; managed gateways	Document recovery limits

RAG Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build multi-format, metadata-rich ingestion	Not Yet Proven	Parser fixtures, metadata schema, durable lifecycle and deletion demo	Python + worker + object storage	LlamaIndex/ Haystack connectors	Link ingestion release

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build, evaluate, and debug hybrid retrieval, reranking, citations, and no-answer behavior	Not Yet Proven	Versioned retrieval/answer evaluation with before-after metrics and failures	Chroma locally; pgvector or Qdrant	Pinecone; Weaviate; OpenSearch; others	Approve vector-store ADR

Agent Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build bounded tool-using workflows with deterministic authorization	Not Yet Proven	LangGraph, typed tools, permission matrix, confirmations, traces	LangGraph + FastAPI policy layer	Direct state machine; PydanticAI; Semantic Kernel	Pass denied-action tests
Evaluate trajectories and recover through retry, clarification, or handoff	Not Yet Proven	Scenario dataset with task, tool, safety, step, and handoff results	Custom harness + traces	LangSmith/Phoenix; human review	Meet trajectory threshold

Backend Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build layered, versioned, validated FastAPI services	Not Yet Proven	Routers, services, repositories, DI, OpenAPI, unit/integration tests	FastAPI + Pydantic	Django/DRF; NestJS; ASP.NET; Spring	Link backend release
Choose REST, SSE, WebSockets, jobs, middleware, and error contracts deliberately	Not Yet Proven	Protocol ADRs and integration tests across success/failure paths	REST + SSE; one WebSocket use case	gRPC; GraphQL; polling; webhooks	Defend protocol choices

Database Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Model transactional SaaS data with migrations, constraints, indexes, and query analysis	Not Yet Proven	ERD, Alembic migrations, integrity tests, query-plan evidence	PostgreSQL + SQLAlchemy + Alembic	MySQL; enterprise DBs; NoSQL	Pass integrity/performance tests
Use Redis only for cache, sessions, limits, temporary state, and coordination	Not Yet Proven	Responsibility matrix, expiry/revocation/rate tests, restart behavior	Redis	Memcached; database sessions; managed cache	Prove no durable truth in Redis

Data Pipelines

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Run durable, idempotent asynchronous document jobs	Not Yet Proven	Queue lifecycle, retry, dead-letter/failure handling, worker-restart evidence	Celery + Redis default	RQ; Dramatiq; ARQ; managed queues; Temporal	Approve worker stack ADR
Maintain data quality, lineage, retention, and repair paths	Not Yet Proven	Data contract, validation report, metadata checks, delete/reprocess procedure	Python pipeline + PostgreSQL metadata	Managed ETL/data platforms	Link data quality report

Evaluation

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Design versioned product-specific evaluation datasets and thresholds	Not Yet Proven	Dataset documentation, representative/adversarial cases, limitations, release gate	Custom harness + pytest	Ragas; DeepEval; Promptfoo; Phoenix	Approve evaluator choice
Measure quality, latency, reliability, security, and cost before release	Not Yet Proven	Automated report, before-after metrics, failure categories, decision	CI evaluation command	Human review; platform evaluations	Link release candidate report

AI Security

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Defend prompt injection, untrusted retrieval, unsafe tools, and output failures	Not Yet Proven	Adversarial suite, schema/tool validation, confirmation and residual-risk notes	Deterministic backend controls + OWASP guidance	Provider guardrails as defense-in-depth	Zero unresolved critical/high failures
Prevent cross-tenant access and sensitive-data leakage	Not Yet Proven	Authorization, vector-filter, object-key, export, trace, secret, and privacy tests	FastAPI policy + PostgreSQL ownership	Managed identity/cloud controls	Link tenant isolation report

SaaS Product Engineering

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Build accounts, workspaces, roles, resource ownership, limits, and admin controls	Not Yet Proven	Multi-user beta, authorization matrix, isolation tests, admin evidence	FastAPI + PostgreSQL	Managed auth/SaaS platforms	Complete external beta

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Implement entitlements, usage ledger, analytics, feedback, and cost controls	Not Yet Proven	Mock ledger, quota tests, event dictionary, feedback loop, budget alerts	PostgreSQL ledger + product events	Stripe/Paddle/etc. after verification	Approve billing decision later

Cloud and DevOps

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Containerize and deploy one complete production path	Not Yet Proven	Docker/Compose, cloud release, environment/secrets contract	Docker + one cloud	Podman; serverless; Kubernetes	Link live release
Automate CI/CD, monitoring, backup, restore, rollback, and incidents	Not Yet Proven	GitHub Actions, alerts, recovery drills, runbook, incident record	GitHub Actions + structured observability	GitLab/Azure/Jenkins; other platforms	Pass restore and rollback drills

Flutter Integration

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Integrate Flutter as a protected production client	Not Yet Proven	Installable app, auth, REST/SSE, history, upload, RAG, agent confirmation tests	Flutter	React Native; native clients; web	Link end to end mobile evidence
Keep secrets, authorization, credentials, and privileged tools server-side	Not Yet Proven	Mobile threat review, secure-token storage, negative tests, failure UX	FastAPI boundary + Flutter secure storage	Managed mobile identity	Pass mobile security review

Product Delivery

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Ship a focused AI SaaS product to external users	Not Yet Proven	Deployment, pilot outcomes, user feedback, support/issues, complete evidence package	Selected production stack	Alternatives recorded in ADR index	Reach pilot acceptance criteria
Improve the product using measured real-user evidence	Not Yet Proven	Decision linked to activation, task success, feedback, latency, cost, or reliability	Analytics + evaluation loop	Qualitative research; experiments	Document next measured iteration

Career Readiness

COMPETENCY	STATUS	EVIDENCE	PRIMARY TOOL USED	ALTERNATIVES UNDERSTOOD	REMAINING GAP
Explain problems, alternatives, architecture, trade-offs, failures, and outcomes professionally	Not Yet Proven	Recorded walkthrough, ADRs, case study, mock interview evidence	Portfolio repositories and diagrams	Alternative architectures understood	Complete interview review
Present five projects as one honest AI Product Engineer profile	Not Yet Proven	Portfolio hub, resume, links, demos, competency audit, next-quarter plan	Evidence-based portfolio	Role-specific presentation variants	No unsupported seniority claims

Final engineer profile

AI	BACKEND + DATA	PRODUCT + PRODUCTION	MOBILE
LLM applications Structured outputs RAG Agents Evaluation AI security	FastAPI REST/streaming PostgreSQL Redis Vector DB Async jobs	SaaS architecture Users/roles/limits Analytics/feedback Docker/cloud/CI/CD Monitoring/recovery	Flutter AI client Streaming UX Files/citations Agent confirmations Production integration

Career outcome

CAREER REALISM Completing this roadmap does not automatically make Muhammad Zaid a senior engineer or guarantee employment at a FAANG company. Successful completion should make him capable of competing for strong junior or early-career AI Application Engineer, LLM Application Engineer, AI Product Engineer, AI Backend Engineer, and Applied AI Engineer roles.

Progression toward senior engineering requires sustained production ownership, real users, team collaboration, code reviews, incidents and recovery, scaling experience, architecture decisions, mentorship, business impact, and several years of professional growth.

REFERENCE INDEX

Reference Index

Official documentation to verify during execution

FRESHNESS NOTE Links checked 22 July 2026. Provider models, free tiers, prices, quotas, privacy terms, and framework APIs change. Re-check official documentation at the start of each phase and record the selected version/configuration in the repository.

- [Gemini API — overview, models, pricing, and limits](#)
- [GroqCloud documentation and active model list](#)
- [OpenRouter documentation, models, and pricing](#)
- [LangChain Python documentation](#)
- [LangGraph overview and Graph API](#)
- [LangSmith observability and evaluation](#)
- [FastAPI official documentation](#)
- [Chroma documentation](#)
- [Qdrant documentation](#)
- [pgvector project documentation](#)
- [PostgreSQL documentation](#)
- [SQLAlchemy documentation](#)
- [Alembic documentation](#)
- [Redis documentation](#)
- [Docker documentation](#)
- [GitHub Actions documentation](#)
- [OWASP GenAI Security Project](#)
- [n8n documentation](#)
- [WhatsApp Business Platform / Cloud API](#)
- [Flutter documentation](#)

Final Note

A final note to Muhammad Zaid

Your advantage is unusual: you already know how to finish and ship products. Use this roadmap to add the AI, backend, data, SaaS, cloud, evaluation, security, and operating depth that turns that strength into AI Product Engineering. Protect the phase order, demand evidence from yourself, and keep the flagship narrow enough to reach real users.

THE STANDARD Muhammad Zaid should not claim a skill because he watched a tutorial, copied a project, or completed a checklist.

A skill is proven only when he can:

- Explain the problem
- Compare valid alternatives
- Choose an appropriate solution
- Defend the decision
- Implement it
- Test it
- Evaluate it
- Secure it
- Deploy it
- Operate it
- Measure it
- Improve it using evidence from real users

This roadmap is not a promise of seniority.

It is a structured path toward becoming a credible AI Product Engineer who understands both the tools he uses and the alternatives he deliberately chooses not to use.